

Introducción a los sistemas operativos

Definición de sistema operativo

Definiremos como sistema operativo al conjunto de programas que controla la ejecución de las aplicaciones y actúa como interfaz entre ellas y el hardware de la computadora.

Objetivos de un sistema operativo

El sistema operativo usualmente se diseña teniendo en cuenta los objetivos siguientes:

- **Comodidad:** todo sistema operativo debe hacer que una computadora sea más cómoda de utilizar.
- **Eficiencia:** significa que un sistema operativo debe lograr que los distintos recursos de un sistema informático se puedan usar de la mejor forma posible.
- **Capacidad de evolución:** todo sistema operativo debe construirse de modo que permita el desarrollo efectivo, la verificación y la introducción de nuevas funciones en el sistema y, a la vez, no interferir en los servicios que brinda.

Funciones de un sistema operativo

Como veremos durante el curso, el kernel es el corazón de cualquier sistema operativo, por lo que todas las funciones vitales deben ser controladas y gestionadas por el propio kernel. Entre las diversas funciones que todo sistema operativo debe cumplir encontramos las siguientes:

- **Gestión de recursos:** Una computadora puede tener a su disposición una gran cantidad de recursos (memoria, espacio de almacenamiento, tiempo de procesamiento, etc.), y los diferentes procesos que se ejecuten en él compiten por ellos. La gestión de recursos es desempeñada por el núcleo o kernel del sistema operativo que debe ocuparse de asignar los recursos de la computadora a los programas en ejecución, así como de evitar que unos programas accedan a los recursos de los otros y de llevar la contabilidad de los recursos consumidos por cada programa. Al gestionar toda la asignación de recursos, el sistema operativo puede implementar políticas que los asignen de forma efectiva y acorde a las necesidades establecidas para dicho sistema.
- **Abstracción:** Un proceso define a qué secciones de memoria puede acceder la aplicación. La principal tarea de un kernel es permitir la ejecución de programas y apoyarlos con características como la abstracción de hardware. Para ejecutar un programa, un núcleo primero configura un espacio de direcciones para el programa, luego carga en memoria el archivo que contiene el código del programa, después configura una pila para el programa y se bifurca a una ubicación determinada dentro del programa, iniciando así finalmente su ejecución. Los programas no deben tener que preocuparse de los detalles de acceso a hardware, o de la configuración particular de una computadora. El sistema operativo se encarga de proporcionar una serie de abstracciones para que los programadores puedan enfocarse en resolver las necesidades particulares de sus usuarios. Un ejemplo de tales

abstracciones es que la información está organizada en archivos y directorios (en uno o muchos dispositivos de almacenamiento).

- **Aislamiento:** En un sistema multiusuario y multitarea cada proceso y cada usuario no tendrá que preocuparse por otros que estén usando el mismo sistema. Idealmente, su experiencia será la misma que si el sistema estuviera exclusivamente dedicado a su atención, aunque fuera un sistema menos poderoso. Para implementar correctamente las funciones de aislamiento hace falta que el sistema operativo utilice hardware específico para dicha protección.
- **Ejecución de servicios para los programas:** Esta función se lleva a cabo en la capa denominada de servicios o llamadas al sistema que permite que la ejecución de los programas sea más cómoda. Una llamada al sistema es un mecanismo utilizado por el proceso para solicitar un servicio del sistema operativo. Para acceder a los servicios proporcionados por el kernel, necesitamos llamar a las funciones del kernel relacionadas. Estas llamadas se encargan de:
 - Ejecutar y destruir procesos.
 - Facilitar la entrada / salida y las operaciones sobre los archivos
 - Detectar y tratar los errores detectados por el hardware.
- **Ejecución de los mandatos de los usuarios:** El sistema operativo debe incorporar un módulo que permita el dialogo entre el usuario y el sistema. Este módulo se denomina Shell y se comporta como un bucle infinito repitiendo la siguiente secuencia:
 1. Espera una orden del usuario.
 2. Una vez recibida una orden la analiza y en caso de ser correcta la ejecuta.
 3. Una vez concluida la orden vuelve a la espera.

Evolución histórica de los sistemas operativos

No se puede comenzar a abordar el tema de los sistemas operativos sin revisar brevemente su desarrollo histórico. Esto no sólo permitirá comprender por qué fueron apareciendo determinadas características y patrones de diseño que se siguen empleando décadas más tarde, sino, adecuar un sistema existente a un entorno cambiante, por mejor diseñado que éste estuviera, lleva casi inevitablemente a abrir espacios de comportamiento no previsto. El espacio más propicio para que prosperen los fallos. Conocer los factores que motivaron a los distintos desarrollos puede ayudar a prever y prevenir problemas. Los podemos relacionar con la evolución tecnológica a lo largo del tiempo y además están íntimamente relacionados con el tipo de procesamiento.

Etapas 1: Proceso en serie.

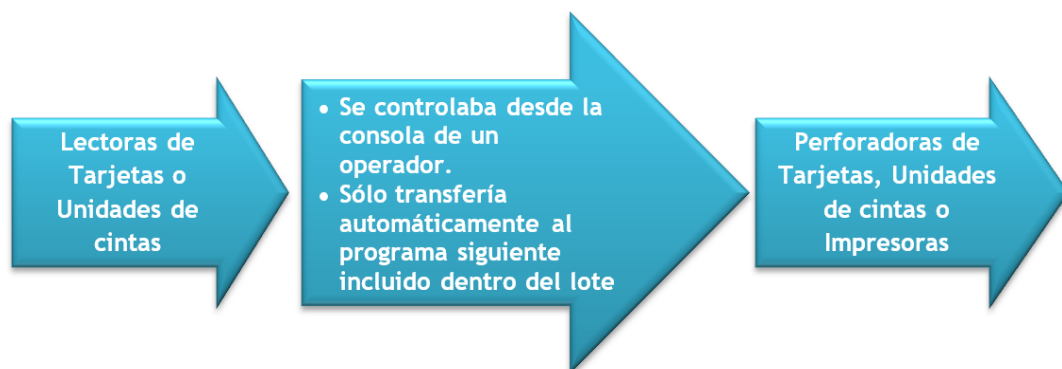
En las primeras computadoras, el programador debía codificar su programa a mano y en instrucciones máquina, debía introducirlo personalmente en la computadora, conectar todos los dispositivos necesarios para la ejecución del programa y desconectarlos al finalizar. Es decir que debía interactuar directamente con el hardware porque no había sistema operativo. Este modo de operación que debía ser realizado por todos los usuarios podría denominarse

proceso en serie porque refleja el hecho de que los usuarios tenían que acceder al computador uno a continuación del otro, es decir, en serie.

Etapa 2: Sistema por lotes monoprogramado.

Servían a máquinas muy grandes que se controlaban desde una única consola y sus dispositivos de entrada eran: lectoras de tarjetas, unidades de cintas, impresoras, perforadoras de tarjetas.

Los antecedentes a lo que hoy se conoce como sistema operativo se pueden encontrar en la automatización inicial del procesamiento de diferentes programas, surgida en los primeros centros de cómputo: cuando en los '50 aparecieron los dispositivos perforadores/lectores de tarjetas de papel, el tiempo que una computadora estaba improductiva esperando a que estuviera listo un programa para poder ejecutarlo disminuyó fuertemente ya que los programadores entregaban su lote de tarjetas perforadas a los operadores, quienes las alimentaban a los dispositivos lectores, que lo cargaban en memoria en un tiempo razonable, iniciaban y monitoreaban la ejecución, y producían los resultados.



En esta primera época en que las computadoras se especializaban en tareas de cálculo intensivo y los dispositivos que interactuaban con medios externos eran prácticamente desconocidos, el rol del sistema monitor o de control era básicamente asistir al operador en la carga de los programas y las bibliotecas requeridas, la comunicación de resultados y la contabilidad de recursos empleados para su cobro.

Los sistemas monitores se hicieron más complejos al implementar protecciones que evitaran la corrupción de otros trabajos (por ejemplo, lanzar erróneamente la instrucción leer siguiente tarjeta causaría que el siguiente trabajo encolado perdiera sus primeros caracteres, corrompiéndolo e impidiendo su ejecución), o que entraran en un ciclo sin fin, estableciendo alarmas que interrumpirían la ejecución de un proceso si éste duraba más allá del tiempo estipulado. Estos monitores implicaban la modificación del hardware para contemplar dichas características de seguridad, de forma que ya se puede hablar de la característica básica de gestión de recursos que caracteriza a los sistemas operativos.

Cabe añadir que el tiempo de carga y puesta a punto de una tarea seguía representando una parte importante del tiempo que la computadora dedicaba al procesamiento: un lector de cintas rápido procesaba del orden de cientos de caracteres por minuto, y a pesar de la lentitud relativa de las computadoras de los '50 ante los estándares de hoy (se medirían por miles de instrucciones por segundo, KHz, en vez de miles de millones como se hace hoy en día, GHz), esperar cinco o diez minutos con el sistema completamente detenido por la carga de un programa moderadamente extenso resulta a todas luces un desperdicio

Estos sistemas presentaban las siguientes características:

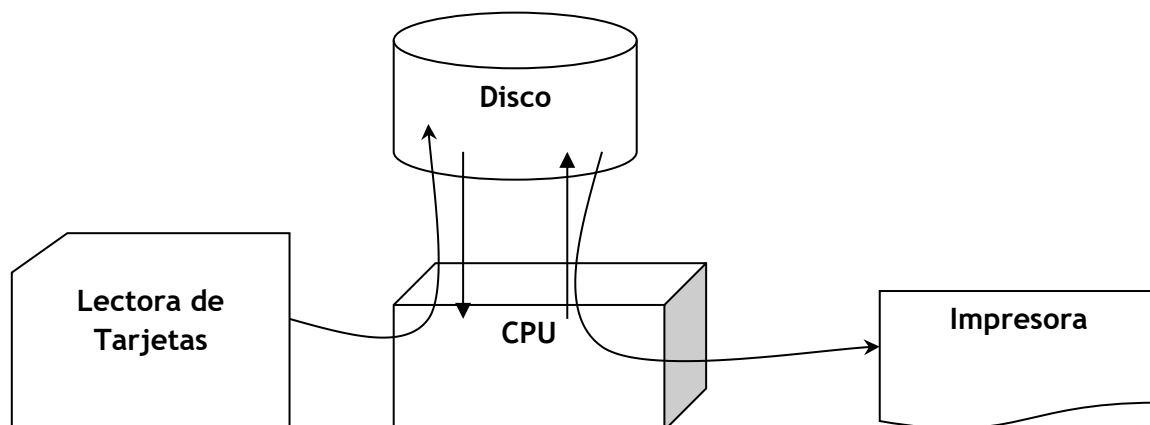
- Agilizaban los procesos agrupando en lotes procesos de características similares.
- En ellos reside un único programa en memoria (monoprogramación), por lo tanto, la CPU quedaba ociosa mucho tiempo, porque los dispositivos de E/S son muy lentos (mecánicos) respecto de la CPU. Además, estos elementos de E/S tenían acceso secuencial. Por ejemplo, la CPU procesaba miles de instrucciones/seg mientras que el lector de tarjetas leía 20 tarjetas/seg.

Etapas 3: Sistema por lotes multiprogramado.

La aparición de los discos agiliza sustancialmente la ejecución de los procesos, porque a diferencia de los elementos de E/S disponibles en la etapa anterior éstos tenían acceso directo y aparece el concepto de SPOOL (Simultaneous Peripheral Operation On Line) que permite que el acceso a los datos no sea desde las lectoras de tarjetas sino desde discos y la salida no sea sobre las impresoras, sino que se grabe una imagen en disco y luego de éste se transfiera a la impresora.

En este caso se utiliza el disco como un gran Buffer y a este tipo de procesamiento se lo llama Spooling, que permite superponer la E/S de un trabajo con el procesamiento de otros, incluso podría estar leyendo la entrada de un trabajo mientras otro imprime y también mientras un trabajo está procesando se puede estar leyendo tarjetas que pasan disco o se puede estar imprimiendo otro trabajo.

Al aparecer el concepto de SPOOL y tener los procesos en disco en lugar de tarjetas o cintas magnéticas el sistema operativo puede planificar el ordenamiento en que ejecutará los procesos, de esta manera el Sistema Operativo puede seleccionar varios procesos para que residan simultáneamente en memoria y de esta manera lograr un mayor aprovechamiento de la CPU pues no existen los largos tiempos de acceso a los periféricos como ocurría en la monoprogramación.



Estos sistemas presentaban las siguientes características:

- Requieren protección de la memoria y del procesador
- Requiere ejecutar instrucciones en modo protegido y otras en modo usuario. Por ejemplo:
 - En modo protegido: Inhibir interrupciones, Modificar el reloj del sistema, Cambiar el mapa de memoria.
 - En modo usuario: Instrucción TRAP.

Etapas 4: Sistemas de tiempo compartido.

En los sistemas de procesamiento batch (por lotes) el usuario no puede interactuar con el programa, en cambio en los sistemas de tiempo compartido el usuario interactúa con el mismo.

El modo de interactuar con las computadoras se modificó drásticamente durante los '60, al extenderse la multitarea para convertirse en sistemas interactivos y multiusuarios, en buena medida diferenciados de los anteriores por la aparición de las terminales, primero teletipos seriales, posteriormente equipos con una pantalla completa como se conocen hasta hoy.

En primer término, la tarea de programación y depuración del código se simplificó fuertemente al poder el programador hacer directamente cambios y someter el programa a la ejecución inmediata. En segundo término, la computadora nunca más estaría simplemente esperando a que esté listo un programa: Mientras un programador editaba o compilaba su programa, la computadora seguía calculando lo que otros procesos requirieran.

Un cambio fundamental entre el modelo de multiprogramación y de tiempo compartido es el tipo de control sobre la multitarea:

- **Multitarea cooperativa o no apropiativa (Cooperative multitasking):** La implementaron los sistemas multiprogramados: Cada proceso tenía control del CPU hasta que éste hacía una llamada al sistema (o indicara su disposición a cooperar por medio de la llamada al sistema). Un cálculo largo no era interrumpido por el sistema operativo, en consecuencia, un error de programador podía congelar la computadora completa.
- **Multitarea preventiva o apropiativa (Preemptive multitasking):** En los sistemas de tiempo compartido, el reloj del sistema interrumpe periódicamente a los diversos procesos, transfiriendo forzosamente el control nuevamente al sistema operativo. El sistema operativo puede entonces elegir otro proceso para continuar la ejecución.

Además, fueron apareciendo de forma natural y paulatina las abstracciones que se conocen hoy en día, como los conceptos de archivos y directorios, y el código necesario para emplearlos iba siendo enviado a las bibliotecas de sistema y, cada vez más hacia el núcleo mismo del sistema operativo.

Un cambio importante entre los sistemas multiprogramados y de tiempo compartido es que la velocidad del cambio entre una tarea y otra es mucho más rápido: si bien en un sistema multiprogramado un cambio de contexto podía producirse sólo cuando la tarea cambiaba de un modo de ejecución a otro, en un sistema interactivo, para dar la ilusión de uso exclusivo de la computadora, el hardware emitía periódicamente al sistema operativo interrupciones que le indicaban que cambie el proceso activo. Diferentes tipos de proceso pueden tener distinto nivel de importancia, ya sea porque son más relevantes para el funcionamiento de la computadora misma (procesos de sistema), porque tienen mayor carga de interactividad (por la experiencia del usuario) o por diversas categorías de usuarios (sistemas con contabilidad por tipo de atención). Esto requiere la implementación de diversas prioridades para cada uno de estos.

Este tipo de sistema se caracteriza por:

- Realizar procesos cortos, asignando pequeñas fracciones de tiempo de procesador a todos los usuarios que lo requieran.

- Acceden usuarios de desde lugares remotos, pudiendo acceder en forma simultánea.
- La prioridad es el tiempo de respuesta de manera tal que el usuario supone que el procesador está dedicado sólo a su tarea.
- El usuario vuelve a tener contacto directo con su proceso a diferencia del procesamiento Batch que solo lo hace el operador a través de la consola.
- Con este tipo de procesamiento aparecen los procesadores de comunicaciones y el concepto de memoria virtual.

Etapa 5: Sistemas de propósito general.

Los sistemas operativos de propósito general eran capaces de operar en lotes, en multiprogramación, en tiempo compartido y en modo multiprocesador. Estos sistemas operativos pusieron entre el usuario y el hardware una gruesa capa de software, de forma que éste sólo veía esta capa, sin tener que preocuparse de los detalles del hardware.

Entre sus inconvenientes se puede destacar su complejo lenguaje de control, que debían aprender los usuarios para preparar sus trabajos, ya que era necesario especificar una inmensa cantidad de detalles y opciones, junto con el gran consumo de recursos que generaban, como ser, los grandes espacios de memoria principal y secundaria ocupados, así como el tiempo de UCP consumido, etc.

Etapa 6: Sistemas de PC, sistemas paralelos, sistemas integrados, sistemas de tiempo real, etc.

Esta etapa de su evolución histórica, que comenzó en la década de 1980, se caracteriza por un desarrollo de los sistemas operativos de propósito general previos, tendiendo a su especialización, a su simplificación y a dar más importancia a la productividad de usuario que al rendimiento de la computadora.

Las características de los sistemas de PC son:

- Cada usuario dispone de su propio computador
- Los primeros equipos no contaban con elementos que protegieran el Sistema Operativo.
- Los primeros equipos eran monotarea y monoprogramados y por lo tanto no permitían cumplir con varias tareas simultáneamente. Sin embargo, los equipos posteriores podían, sin ser multiprogramados, pero tenían la facilidad del SPOOL, estar imprimiendo un texto y a la vez trabajar sobre otro. Tanto unos como otros no tenían entorno gráfico.
- Al integrarse en red aumentaba su potencia.
- La seguridad de la información en disco fue evolucionando con los sistemas operativos (FAT - NTFS).

Los sistemas paralelos tienen varios procesadores y permiten el multiprocesamiento, es decir estar procesando simultáneamente varios procesos. Estos sistemas pueden ser:

- Sistemas de multiprocesadores que tienen las siguientes características:
 - Consiste en un sistema con varios procesadores que comparten la memoria, el bus o sistema de interconexión, etc.

- Aumenta el número de trabajos en ejecución.
- Sistemas de multicomputadoras o distribuidos que presentan las siguientes características:
 - Tienen varios procesadores donde cada uno tiene su propia memoria, bus, etc.
 - Las computadoras suelen estar unidas mediante una red, de forma que puedan compartir información y periféricos.
 - Comparten todos los recursos.
 - Aumentan el número de trabajos en ejecución.
 - Son más confiables, pues si una queda fuera de servicio el resto sigue trabajando normalmente.

Los sistemas de tiempo real son sistemas de propósito especial, por ejemplo, monitorear dispositivos, analizar datos o tomar acciones en tiempo muy corto. Sus aplicaciones pueden ser: control de robots, control de centrales nucleares, control de aeronaves.

Los sistemas operativos integrados operan en las computadoras que controlan dispositivos como por ejemplo motores de automóvil y brazos robóticos, que no aceptan software instalado por el usuario. Estos sistemas suelen tener tareas muy específicas por lo que proporcionan servicios muy limitados.

Componentes de un sistema operativo

Se suele considerar que un sistema operativo está formado por tres capas: el núcleo, los servicios y el intérprete de mandatos o Shell.

El núcleo es la parte del sistema operativo que interacciona directamente con el hardware de la máquina. Las funciones del núcleo se centran en la gestión de recursos, como el procesador, tratamiento de interrupciones y las funciones básicas de manipulación de memoria.

Estas funciones las ejecutan distintos elementos del sistema operativo que suelen agruparse según su objetivo en distintos grupos, cada uno de cuales se ocupa de las siguientes tareas:

- **Gestión de los procesos:** Usualmente los sistemas operativos proveen las siguientes funciones para la gestión de procesos: creación y finalización de procesos, planificación y activación de procesos, sincronización de procesos, comunicación entre procesos, administración de los bloques de control de proceso.
- **Gestión de la memoria:** Los componentes del sistema operativo que administran al sistema de memoria se encargan de: asignar memoria, liberar memoria, compartir memoria.
- **Gestión de las operaciones de Entrada / Salida:** El gestor de E/S debe controlar el funcionamiento de todos los dispositivos de E/S para facilitar el manejo de los dispositivos periféricos y debe ofrecer mecanismos de protección que impidan a los usuarios acceder sin control a los dispositivos periféricos.
- **Gestión del almacenamiento secundario o los archivos y directorios:** El gestor de archivos es la parte del sistema operativo que tiene como objetivo tanto el de facilitar el

manejo de los dispositivos periféricos ofreciendo una visión lógica simplificada de los mismos en forma de archivos, como el de proteger a los usuarios, poniendo limitaciones a los archivos que es capaz de manipular cada usuario.

- **Funciones de soporte:** Dentro de este grupo se encuentran aquellos elementos que realizan las funciones siguientes: gestionar las interrupciones, administrar las auditorías, administrar las protecciones y la seguridad.

El sistema operativo ofrece a los programas un conjunto de servicios que pueden ser solicitados cuando sea necesario, proporcionando a los programas una visión de computadora. Dentro de estos servicios podemos mencionar brevemente a: la ejecución de programas, el acceso a los dispositivos de E/S, el acceso controlado a los archivos, el acceso al sistema, la detección y respuesta a errores, la contabilidad del sistema.

La mayoría de los usuarios de computadoras habrán tenido cierta experiencia con un sistema operativo como Windows, Linux, FreeBSD o Mac OS X, pero las apariencias pueden engañar. El programa con el que los usuarios generalmente interactúan se denomina Shell, cuando está basado en texto, y GUI (Graphical User Interface; Interfaz gráfica de usuario) cuando utiliza elementos gráficos o iconos. En realidad, no forma parte del sistema operativo, aunque lo utiliza para llevar a cabo su trabajo.

El Shell es el nivel más bajo del software en modo usuario y permite la ejecución de otros programas, como un navegador Web, lector de correo electrónico o reproductor de música, programas que también utilizan en forma intensiva al sistema operativo.

Arquitectura de los sistemas operativos

Un sistema operativo es un conjunto de programas grande y complejo que está compuesto, por una serie de componentes con funciones bien definidas. Cada sistema operativo estructura estos componentes de distinta forma. En función de esta estructura se pueden agrupar los sistemas operativos en dos grandes grupos: sistemas operativos monolíticos y sistemas operativos estructurados. Dentro del segundo de los grupos se pueden agrupar los sistemas operativos con los diseños siguientes: sistemas de capas, microkernels, máquinas virtuales, sistemas cliente-servidor y exokernels. De ninguna manera esta lista quiere sugerir que estas arquitecturas sean exhaustivas, simplemente nos dan una idea de algunos diseños que se han probado en la práctica.

Monolítica

Un sistema operativo de este tipo no tiene una estructura clara y bien definida. Todos sus componentes se encuentran integrados en un único programa (el sistema operativo) que ejecuta en un único espacio de direcciones. En este tipo de sistemas todas las funciones que ofrece el sistema operativo se ejecutan en un modo núcleo.

Estos sistemas operativos han surgido, normalmente, de sistemas operativos sencillos y pequeños a los que se les ha ido añadiendo un número mayor de funcionalidades. Esto les ha hecho evolucionar y crece hasta convertirlos en programas grandes y complejos formados por muchas funciones situadas todas ellas en un mismo nivel. Ejemplos claros de este tipo de sistemas son MS-DOS y UNIX. Ambos comenzaron siendo pequeños sistemas operativos, que fueron haciéndose cada vez más grandes debido a la gran popularidad que adquirieron.

El problema que plantean este tipo de sistemas radica en lo complicado que es modificar el sistema operativo para añadir nuevas funcionalidades y servicios. En efecto, añadir una nueva característica al sistema operativo implica la modificación de un gran programa, compuesto por miles de líneas de código fuente y funciones, cada una de las cuales puede invocar a otras cuando así lo requiera. Además, en este tipo de sistemas no se sigue el principio de ocultación de la información. Para solucionar este problema es necesario dotar de cierta estructura al sistema operativo.

Todos los procesos son ejecutados en el menor nivel y con la siguiente secuencia:

- Llamada al Kernel
- Examina parámetros
- Consulta tablas con apuntadores
- Ejecuta
- Devuelve el control al programa usuario.

Por capas

El enfoque monolítico suele denominarse sistema fuertemente acoplado porque los cambios en una parte del sistema pueden tener efectos de gran alcance en otras partes. Otra posibilidad es diseñar un sistema débilmente acoplado. Un sistema de este tipo se divide en componentes separados y más pequeños que tienen una funcionalidad específica y limitada. Todos estos componentes juntos forman el núcleo. La ventaja de este enfoque modular es que los cambios en un componente afectan sólo a ese componente, y no a otros, lo que permite a los implementadores del sistema más libertad a la hora de crear y cambiar el funcionamiento interno del sistema.

Un sistema puede modularizarse de muchas maneras. Una de ellas es el enfoque por capas, en el que el sistema operativo se organiza como una jerarquía de capas (niveles), donde cada capa ofrece una interfaz clara y bien definida a la capa superior y solamente utiliza los servicios que le ofrece la capa inferior. La capa inferior (capa 0) es el hardware; la superior (capa N) es la interfaz de usuario. Esta estructura de capas se ejemplifica con un modelo de 6 capas en la figura siguiente.

Una capa del sistema operativo es una implementación de un objeto abstracto compuesto por datos y las operaciones que pueden manipular esos datos. Una capa típica del sistema operativo, por ejemplo, la capa M, consiste en estructuras de datos y un conjunto de funciones que pueden ser invocadas por capas de nivel superior. La capa M, a su vez, puede invocar operaciones en capas de nivel inferior.

La principal ventaja del enfoque por capas es la sencillez de construcción y depuración. Las capas se seleccionan de modo que cada una de ellas utilice funciones (operaciones) y servicios sólo de capas de nivel inferior. Este enfoque simplifica la depuración y la verificación del sistema. La primera capa puede depurarse sin preocuparse del resto del sistema, ya que, por definición, sólo utiliza el hardware básico (que se supone correcto) para implementar sus funciones. Una vez depurada la primera capa, se puede suponer su correcto funcionamiento mientras se depura la segunda capa, y así sucesivamente. Si se encuentra un error durante la depuración de una capa en particular, el error debe estar en esa capa, porque las capas

por debajo de ella ya están depuradas. De este modo, el diseño y la implementación del sistema se simplifican.

Cada capa se implementa sólo con operaciones proporcionadas por capas de nivel inferior. Una capa no necesita saber cómo se implementan estas operaciones; sólo necesita saber qué hacen estas operaciones. Por lo tanto, cada capa oculta la existencia de determinadas estructuras de datos, operaciones y hardware a las capas de nivel superior.

Los sistemas por capas se han utilizado con éxito en redes informáticas (como TCP/IP) y aplicaciones web. Sin embargo, son relativamente pocos los sistemas operativos que utilizan un enfoque puramente por capas. Uno de los motivos es la dificultad de definir adecuadamente la funcionalidad de cada capa. Además, el rendimiento general de estos sistemas es bajo debido a la sobrecarga que supone que un programa de usuario atraviese varias capas para obtener un servicio del sistema operativo. Sin embargo, en los sistemas operativos actuales es habitual que exista cierta estratificación. Por lo general, estos sistemas tienen menos capas con más funcionalidad, lo que proporciona la mayoría de las ventajas del código modularizado al tiempo que evita los problemas de definición e interacción de capas.



Este enfoque lo utilizó por primera vez el sistema operativo THE (Dijkstra, 1968), un sistema operativo sencillo que estaba formado por seis capas, como se muestra en la siguiente tabla. Otro ejemplo de sistema operativo diseñado por capas es el OS/2 (Deitel, 1994), descendiente de MS-DOS.

Microkernels

A medida que el sistema operativo se expande, el kernel se hace más grande y difícil de gestionar. A mediados de los años 80 se desarrolló un sistema operativo que modularizaba el kernel. El enfoque consiste en implementar la mayor parte de los servicios y funciones del sistema operativo en procesos de usuario, dejando solo una pequeña parte del sistema operativo ejecutando en modo núcleo. Mientras que a esta parte se la llama *microkernel*, a

los procesos que ejecutan el resto de las funciones se les denomina *servidores*. Normalmente los microkernels proporcionan una gestión de la memoria y de los procesos mínima, además de un mecanismo de comunicaciones entre el programa cliente y los distintos servicios que se ejecutan también en el espacio de usuario.

La siguiente tabla presenta la estructura de un sistema operativo con estructura cliente-servidor. Como puede apreciarse en la tabla, el sistema operativo está formado por diversas partes, cada una de las cuales puede desarrollarse por separado.

Procesos Cliente		Procesos Servidores						Modo usuario
Programa de usuario	Programa de usuario	Servidor de procesos	Servidor de memoria	Servidor de E/S	Servidor de archivos y directorios	Servidor de seguridad	Servidor de comunicaciones	
API ¹	API							
Micronúcleo								Modo núcleo
Hardware								

No hay una definición clara de las funciones que debe llevar a cabo un microkernel. La mayoría incluyen la gestión de interrupciones, gestión básica de procesos y de memoria y servicios básicos de comunicación entre procesos. Para solicitar un servicio en este tipo de sistemas, como por ejemplo la creación de un proceso, el proceso de usuario, proceso denominado *cliente*, solicita el servicio al *servidor* del sistema operativo correspondiente, en este caso al servidor de procesos. A su vez, el proceso servidor puede requerir los servicios de otros servidores, como es el caso del servidor de memoria. En este caso, el servidor de procesos se convierte en cliente del servidor de memoria.

La ventaja de este modelo es la gran flexibilidad que presenta. Cada proceso servidor sólo se ocupa de una funcionalidad concreta, lo que hace que cada parte pueda ser pequeña y manejable. Esto a su vez facilita el desarrollo y depuración de cada uno de los procesos servidores. Por otro lado, el ejecutar los servicios en espacios separados aumenta la fiabilidad del conjunto, puesto que un fallo solamente afecta a un módulo.

Otra ventaja del método de microkernel es la facilidad para ampliar el sistema operativo. Todos los servicios nuevos se añaden al espacio de usuario y, en consecuencia, no requieren que se modifique el kernel. Cuando surge la necesidad de modificar el kernel, los cambios tienden a ser pocos, porque el microkernel es un kernel muy pequeño. El sistema operativo resultante es más fácil de portar de un diseño hardware a otro. El microkernel también proporciona más seguridad y fiabilidad, dado que la mayor parte de los servicios se ejecutan como procesos de usuario, en lugar de como procesos del kernel. Si un servicio falla, el resto del sistema operativo no se ve afectado. En cuanto a las desventajas, podemos mencionar que estos sistemas presentan una mayor sobrecarga en el tratamiento de los servicios que los sistemas monolíticos. Esto se debe a que los distintos componentes de un sistema operativo de este tipo ejecutan en espacios de direcciones distintos, lo que hace que su activación requiera más tiempo.

¹ API: Application Programming Interface.

Minix [Tanenbaum, 1998], Mach [Accetta, 1986] y Amoeba [Mullender, 1990] son ejemplos de sistemas operativos que siguen este modelo. Windows NT también sigue esta filosofía de diseño, aunque muchos de los servidores (el gestor de procesos, gestor de E/S, gestor de memoria, etc.) se ejecutan en modo núcleo por razones de eficiencia.

En definitiva, se caracterizan por:

- Ser fácilmente adaptables a sistemas distribuidos
- Tratan de llevar el código a las capas más altas
- El kernel solo maneja la comunicación entre clientes y servidores
- Los clientes y servidores se ejecutan en modo usuario y no tienen acceso directo al hardware.

Sistemas operativos modulares

Tal vez la mejor metodología actual para el diseño de sistemas operativos implique el uso de módulos cargables del núcleo (LKMs: Loadable Kernel Modules). En este caso, el núcleo tiene un conjunto de componentes básicos y puede enlazar servicios adicionales a través de módulos, ya sea en tiempo de arranque o durante el tiempo de ejecución. Este tipo de diseño es común en las implementaciones modernas de UNIX, como Linux, macOS y Solaris, así como en Windows.

La idea del diseño es que el núcleo proporcione servicios básicos, mientras que otros servicios se implementan dinámicamente, a medida que el núcleo se ejecuta. Vincular servicios dinámicamente es preferible a añadir nuevas características directamente al núcleo, lo que requeriría recompilarlo cada vez que se realizara un cambio. Así, por ejemplo, podríamos construir algoritmos de planificación de CPU y gestión de memoria directamente en el núcleo y luego añadir soporte para diferentes sistemas de archivos mediante módulos cargables.

El resultado global se asemeja a un sistema por capas en que cada sección del núcleo tiene interfaces definidas y protegidas; pero es más flexible que un sistema por capas, porque cualquier módulo puede llamar a cualquier otro módulo. El enfoque también se parece al del micronúcleo en que el módulo principal sólo tiene funciones básicas y conocimientos sobre cómo cargar y comunicarse con otros módulos; pero es más eficiente, porque los módulos no necesitan invocar el paso de mensajes para comunicarse.

Linux utiliza módulos del kernel cargables, principalmente para dar soporte a controladores de dispositivos y sistemas de archivos. Los LKM pueden insertarse en el kernel cuando el sistema se inicia (o arranca) o durante el tiempo de ejecución, como cuando se conecta un dispositivo USB a una máquina en funcionamiento. Si el núcleo de Linux no dispone del controlador necesario, puede cargarse dinámicamente. Los LKM también pueden eliminarse del kernel durante el tiempo de ejecución. Para Linux, los LKMs permiten un kernel dinámico y modular, manteniendo las ventajas de rendimiento de un sistema monolítico.

Sistemas operativos híbridos

En la práctica, muy pocos sistemas operativos adoptan una estructura única y estrictamente definida. En su lugar, combinan diferentes estructuras, dando lugar a sistemas híbridos que abordan cuestiones de rendimiento, seguridad y usabilidad. Por ejemplo, Linux es monolítico, porque tener el sistema operativo en un único espacio de direcciones proporciona un

rendimiento muy eficiente. Sin embargo, también es modular, de modo que pueden añadirse dinámicamente nuevas funcionalidades al núcleo. Windows también es monolítico en gran medida (de nuevo, principalmente por motivos de rendimiento), pero conserva algunos comportamientos típicos de los sistemas de micronúcleo, como la compatibilidad con subsistemas independientes, conocidos como personalidades del sistema operativo, que se ejecutan como procesos en modo usuario. Los sistemas Windows también ofrecen soporte para módulos de kernel cargables dinámicamente. Otros tres ejemplos de sistemas híbridos son el sistema operativo macOS de Apple y los dos sistemas operativos móviles más destacados: iOS y Android.

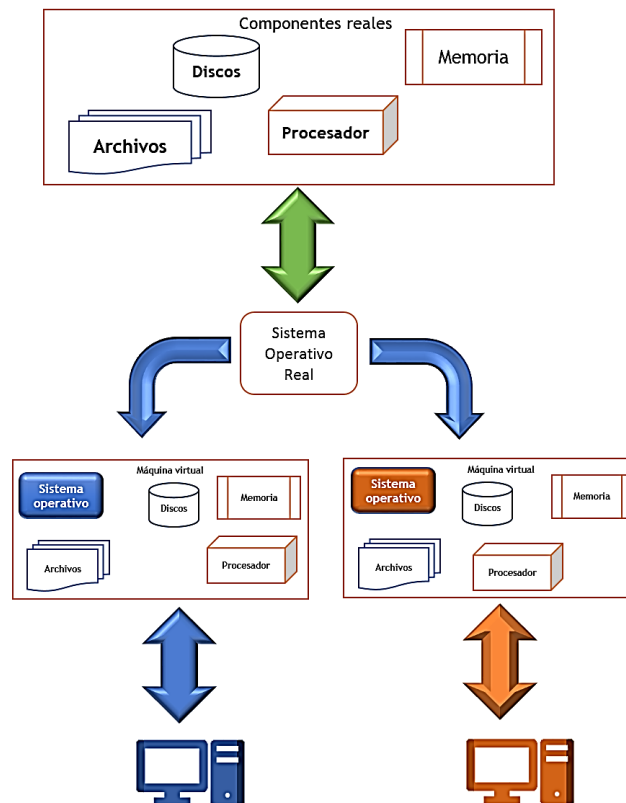
Nanokernel

Un nanokernel es un núcleo que puede delegar prácticamente todos los servicios, desde los más básicos como los controladores de interrupciones o el temporizador, hasta controladores de dispositivos para que la demanda de memoria del núcleo sea aún menor que la de un micronúcleo convencional.

Los nanokernels son núcleos comparativamente pequeños que ofrecen abstracción de hardware; sin embargo, carecen de servicios de sistema. Proporcionan las funciones básicas del sistema operativo y el resto se implementa como aplicaciones. Este tipo de kernels se utilizan para crear sistemas operativos para dispositivos muy específicos en el ámbito militar, científico o industrial.

Máquinas virtuales

La principal de una máquina virtual es la de abstraer el hardware de la computadora, formando así varios entornos de ejecución diferentes, creando de esta manera la ilusión de que cada entorno de ejecución está operando en su propia computadora personal.



Con los mecanismos de planificación de la CPU y las técnicas de memoria virtual, un sistema operativo puede crear la ilusión de que un proceso tiene su propio procesador con su propia memoria (virtual). Usualmente, un proceso utiliza características adicionales, tales como llamadas al sistema y un sistema de archivos, que el hardware básico no proporciona. El método de máquina virtual no proporciona ninguna de estas funcionalidades adicionales, sino que proporciona una interfaz que es idéntica al hardware básico subyacente. Cada proceso dispone de una copia (virtual) de la computadora subyacente.

Exokernels

Esta arquitectura, en vez de clonar la máquina actual, como se hace con las máquinas virtuales, la particiona, es decir que a cada usuario se le proporciona un subconjunto de los recursos. Así, una máquina virtual podría obtener los bloques de disco del 0 al 1023, la siguiente podría obtener los bloques de disco del 1024 al 2047 y así sucesivamente. En la capa inferior, que se ejecuta en el modo kernel, hay un programa llamado exokernel. Su trabajo es asignar recursos a las máquinas virtuales y después comprobar los intentos de utilizarlos, para asegurar que ninguna máquina trate de usar los recursos de otra. Cada máquina virtual de nivel de usuario puede ejecutar su propio sistema operativo, al igual que en la VM/370 y las Pentium 8086 virtuales, con la excepción de que cada una está restringida a utilizar sólo los recursos que ha pedido y que le han sido asignados. La ventaja del esquema del exokernel es que ahorra una capa de asignación. En los otros diseños, cada máquina virtual piensa que tiene su propio disco, con bloques que van desde 0 hasta cierto valor máximo, por lo que el monitor de la máquina virtual debe mantener tablas para reasignar las direcciones del disco (y todos los demás recursos). Con el exokernel, esta reasignación no es necesaria. El exokernel sólo necesita llevar el registro para saber a cuál máquina virtual se le ha asignado cierto recurso. Este método sigue teniendo la ventaja de separar la multiprogramación (en el exokernel) del código del sistema operativo del usuario (en espacio de usuario), pero con menos sobrecarga, ya que todo lo que tiene que hacer el exokernel es mantener las máquinas virtuales separadas unas de las otras.

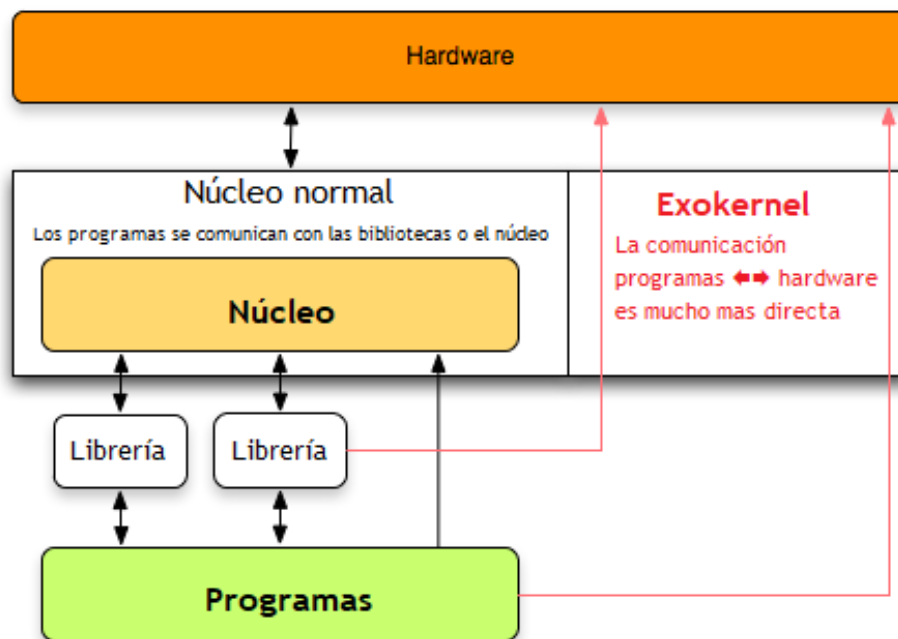
Los SO tradicionales siempre repercuten en el rendimiento, la funcionalidad y el área de las aplicaciones que se construyen sobre ellos, porque el SO se sitúa entre las aplicaciones de usuario y los recursos de hardware, y generalmente presentan recursos de hardware a las aplicaciones a través de abstracciones de alto nivel, como los sistemas de archivos virtuales. El sistema operativo exokernel intenta resolver este problema eliminando la noción de que un SO debe proporcionar abstracciones sobre las que construir programas. La idea es imponer a los desarrolladores el menor número posible de abstracciones y darles la libertad de utilizarlas cuando las necesiten, permitiéndoles tomar tantas decisiones como sea posible sobre abstracciones de hardware.

El principal objetivo de un exokernel es garantizar que no haya ninguna abstracción forzada, que es lo que diferencia a un exokernel de los kernels micro y monolíticos. Los exokernels son diminutos, ya que la funcionalidad se limita a asegurar la protección y multiplexación de recursos, lo cual es considerablemente más simple que la implementación de microkernels convencionales de paso de mensajes y la implementación de kernels monolíticos de abstracciones de alto nivel.

Los exokernels no obligan a un programa de aplicación a superponer sus abstracciones sobre otras abstracciones que fueron diseñadas con diferentes requisitos en mente. Por ejemplo,

en el proyecto M.I.T. Exokernel, el servidor web Cheetah almacena paquetes de protocolo de Internet preformateados en el disco, el kernel proporciona acceso seguro al disco al evitar la lectura y escritura no autorizadas, pero la forma en que se abstrae el disco depende de la aplicación o de las bibliotecas que utiliza la aplicación.

Las aplicaciones implementadas se denominan sistemas operativos de biblioteca; pueden solicitar direcciones de memoria específicas, bloques de disco, etc. El kernel solo garantiza que el recurso solicitado esté libre y que la aplicación pueda acceder a él. Este acceso de hardware de bajo nivel permite al programador implementar abstracciones personalizadas y omitir las innecesarias, generalmente para mejorar el rendimiento de un programa. También permite a los programadores elegir qué nivel de abstracción quieren, alto o bajo.



Algunas de las características de los sistemas operativos exokernel son:

- Soporte superior para el control de aplicaciones
- Divide la seguridad de la gestión
- Las abstracciones se trasladan de forma segura a un sistema operativo de biblioteca no fiable
- Proporciona una interfaz de bajo nivel
- Los sistemas operativos de biblioteca proporcionan portabilidad y compatibilidad

Las ventajas del sistema operativo exokernel incluyen:

- Mejora del rendimiento de los programas
- Uso más eficiente de los recursos de hardware mediante la asignación y revocación precisa de recursos
- Desarrollo y pruebas más sencillos de nuevos sistemas operativos
- Cada programa del espacio de usuario puede aplicar su propia gestión optimizada de la memoria.

Algunos de los inconvenientes del sistema operativo exokernel incluyen:

- Menor coherencia
- Diseño complejo de la interfaz del exokernel

Un ejemplo de sistema operativo exokernel es Némesis, escrito por la Universidad de Cambridge, la Universidad de Glasgow, Citrix Systems y el Instituto Sueco de Ciencias de la Computación. El M.I.T. también ha construido varios sistemas basados en exokernel, incluido ExOS. Aunque el concepto de exokernel existe al menos desde 1994, hoy en día los exokernels son todavía un esfuerzo de investigación y no se han utilizado en ningún sistema operativo comercial importante.

Llamadas al sistema operativo

La capa de servicios, o de llamadas al sistema, ofrece a los procesos unos servicios a través de una interfaz de programación o API (Application Programming Interface). Desde el punto de vista de los procesos esta capa extiende la funcionalidad del computador, por lo que se suele decir que el sistema operativo ofrece una máquina extendida a los procesos. Son la interfaz entre un proceso y el sistema operativo y están disponibles como instrucciones en lenguajes ensambladores o de alto nivel. En estos últimos semejan llamadas predefinidas a funciones o subrutinas. De esta forma se facilita la elaboración de los programas, puesto que se apoyan en las funciones que les suministra el sistema operativo. Como ejemplos de llamadas podemos mencionar: abrir archivo, cerrar archivo, crear archivo, leer archivos, escribir archivos, mensaje de error.

Tipos de llamadas al sistema

- Para control de procesos: finalizar proceso, abortar proceso, cargar proceso, ejecutar proceso, crear proceso, obtener y/o establecer atributos de procesos, asignar o liberar memoria, esperar un suceso
- Para manipulación de archivos: crear o eliminar archivos, abrir o cerrar archivos, leer o escribir archivos, obtener o establecer atributos
- Manipulación de dispositivos: solicitar y liberar dispositivos, reposicionar sobre un dispositivo, conectar y desconectar lógicamente dispositivos, obtener y establecer atributos
- Mantenimiento de información: obtener y establecer la fecha y la hora, obtener y establecer los atributos de un proceso, dispositivo y/o archivo, obtener y establecer atributos del sistema
- Comunicaciones: crear o eliminar conexión, enviar y recibir mensajes, transferir información de estado, conectar o desconectar dispositivos remotos.

Seguridad y los mecanismos de protección

La seguridad incluye 2 cuestiones: una busca garantizar la identidad de los usuarios y la otra se ocupa de definir lo que puede hacer cada uno de ellos. La primera de las cuestiones es desarrollada bajo el concepto de autenticación, mientras que la otra está vinculada a los privilegios. La seguridad es una de las funciones del sistema operativo que, para llevarla a

cabo, se ha de basar en los mecanismos de protección que le proporciona el hardware. Estos mecanismos son los siguientes: protección del procesador, protección de E/S, protección de memoria.

Protección del procesador

Muchos procesadores proporcionan al menos dos modos de ejecución. Ciertas instrucciones se pueden ejecutar en modos privilegiados únicamente. Éstas incluirían lectura y modificación de los registros de control, por ejemplo, la palabra de estado de programa; instrucciones de E/S primitivas; e instrucciones relacionadas con la gestión de memoria. Adicionalmente, ciertas regiones de memoria sólo se pueden acceder en los modos más privilegiados.

El modo menos privilegiado a menudo se denomina modo usuario, porque los programas de usuario típicamente se ejecutan en este modo. El modo más privilegiado se denomina modo núcleo o modo kernel (y también modo control, modo monitor, modo sistema, modo supervisor o modo privilegiado). El término modo núcleo se refiere al núcleo del sistema operativo, que es la parte del sistema operativo que engloba las funciones más importantes del sistema.

El motivo por el cual se usan varios modos es claro. Se necesita proteger al sistema operativo y a las tablas clave del sistema, por ejemplo, los bloques de control de proceso, de la interferencia con programas de usuario. En modo núcleo, el software tiene control completo del procesador y de sus instrucciones, registros, y memoria. Este nivel de control no es necesario y por seguridad no es recomendable para los programas de usuario CPU. El cambio de modo de operación se realiza a través de interrupciones.

Cuando el hardware detecta que un programa en modo usuario intenta ejecutar una instrucción no permitida, o acceder directamente a un periférico o una dirección fuera del mapa de memoria, genera una excepción de error. Dicha excepción activa al sistema operativo, que decide la acción a tomar.

Los programas de usuario deberán ejecutar en el modo más restrictivo para evitar que puedan interferir unos con otros. Es absolutamente imprescindible evitar que un programa de usuario pueda poner el procesador en modo privilegiado, puesto que se perdería la protección. Por ello, no existe ninguna instrucción en lenguaje ensamblador que cambie directamente los bits de modo de ejecución de usuario a privilegiado. Sin embargo, existe la instrucción máquina inversa, que cambia de modo privilegiado a modo usuario. Esta instrucción, que suele ser el RETI (retorno de interrupción), es utilizada por el sistema operativo antes de dejar que ejecute un programa de usuario.

El único mecanismo que tiene el procesador para pasar de modo usuario a modo privilegiado es la interrupción. Por lo tanto, si queremos garantizar la protección es imprescindible cumplir que:

- Todas las direcciones de la tabla de interrupciones apunten a rutinas del sistema operativo, puesto que es el único que debe ejecutar en modo privilegiado.
- Los programas de usuario no puedan modificar dicha tabla.

Para que un programa de usuario, que ejecuta en modo usuario, pueda activar al sistema operativo, que ejecuta en modo privilegiado, existe la instrucción en lenguaje ensamblador

de llamada al sistema. Dicha instrucción produce un ciclo de aceptación de interrupción, por lo que indirectamente pone al procesador en modo privilegiado y salta al sistema operativo. De forma genérica se llama a esa instrucción TRAP.

Protección de E/S

La protección de la entrada/salida es fundamental para evitar que unos usuarios puedan acceder indiscriminadamente a los periféricos. Uno de los objetivos prioritarios es la protección de los periféricos de almacenamiento. Un controlador de disco no entiende de archivos ni de derechos de acceso, sólo de sectores. Por lo tanto, un programa que pueda acceder directamente a dicho controlador puede leer y escribir en cualquier parte del dispositivo. Prohibiendo el acceso de los programas de usuario al mapa de E/S se evita dicho problema. Cuando un programa necesite acceder a un periférico deberá solicitárselo al sistema operativo, que previamente se asegurará de que la operación está permitida. Dependiendo de que la arquitectura del procesador sea de mapa de E/S propio o de mapa de E/S común con el mapa de memoria, la protección se realiza por mecanismos distintos.

Cuando existe un mapa de E/S propio, el procesador cuenta con instrucciones de máquina específicas para acceder a ese mapa. Estas son las instrucciones de E/S. La protección viene garantizada porque dichas instrucciones solamente son ejecutables en modo privilegiado. Si un programa de usuario las intenta ejecutar el procesador produce una excepción hardware síncrona de código de instrucción inválido.

Cuando el mapa de E/S es común con el mapa de memoria, el acceso se realiza mediante las instrucciones máquina de LOAD y STORE, que están permitidas en modo usuario. La protección se basa, en este caso, en proteger ese espacio del mapa de memoria, de forma que los usuarios no lo puedan acceder.

Protección de memoria

Los mecanismos de protección de memoria deben evitar que un programa en ejecución direcciona a posiciones de memoria que no le hayan sido asignadas por el sistema operativo.

Una solución empleada en algunas máquinas que no tienen memoria virtual, es decir en las máquinas de memoria real, consiste en incluir una pareja de registros valla (límite y base). En esta solución se le asigna al programa una zona de memoria contigua. Todos los direccionamientos se calculan sumando el contenido del registro base, de forma que para el programa es como si estuviese cargado a partir de la posición «0» de memoria. Además, en cada acceso un circuito comparador comparará la dirección generada con el valor del registro límite.

En caso de desbordamiento, el comparador genera una excepción hardware síncrona de violación de memoria, para que el sistema operativo trate el tema, lo que, en la práctica, supone que parará el programa produciendo un volcado de memoria que consiste en almacenar en disco el contenido de la memoria. Posteriormente, se puede determinar la razón del fallo analizando el volcado de memoria mediante una adecuada herramienta de depuración.

En los sistemas con memoria virtual existen dos mecanismos de protección de memoria. El primero se basa en que en modo privilegiado se puede direccionar todo el mapa de memoria virtual, mientras que en modo usuario solamente se puede direccionar una parte del mapa,

por ejemplo, las direcciones que empiezan con 0, produciéndose una excepción hardware síncrona de violación de memoria en caso de que se genere una dirección no permitida, por ejemplo, que la dirección empiece con 1. En otras palabras, mientras que en nivel de núcleo el procesador permite acceder a todo el mapa de memoria virtual, en el nivel de usuario el procesador sólo permite direccionar a una parte del mapa, en el ejemplo del párrafo anterior serían sólo a las direcciones que empiezan por 0. La MMU generará una excepción de violación de memoria en caso de que en nivel de usuario se genere una dirección no permitida, como por ejemplo las que empiecen con 1.

El segundo consiste en dotar, a cada proceso de su propia tabla de páginas, que se selecciona mediante el registro RIED (Registro de Identificador de Espacio de Direccionamiento) y especifica los espacios de memoria asignados por el sistema operativo a ese proceso. La tabla de páginas incorpora información de protección tanto a nivel de región como de página. Típicamente se pueden especificar permisos de lectura, escritura y ejecución. De esta forma se consigue que cada proceso disponga de su propio espacio virtual, por lo que no puede acceder a los espacios de memoria de los otros procesos.

La MMU, al mismo tiempo que realiza la traducción de cada dirección, comprueba que no se sobrepase el límite de ninguna de las tablas involucradas. Si esto ocurre significa que la dirección es correcta, es decir, que está dentro de las regiones asignadas al proceso y que el acceso solicitado está permitido (lectura, escritura o ejecución). En caso de sobrepasarse, o de no ser un acceso permitido, generará una excepción de violación de memoria, para que el sistema operativo realice la acción correctora oportuna.

Activación del sistema operativo

El trabajo del sistema operativo puede provenir de las siguientes fuentes:

- Llamadas al sistema emitidas por los programas.
- Interrupción producida por los periféricos.
- Condiciones de excepción o error del hardware.

En todos estos casos se deja de ejecutar el proceso en ejecución y se pasa a ejecutar el sistema operativo. Los mecanismos para romper la sentencia lineal de ejecución son dos: las instrucciones de bifurcación y las interrupciones.

El primero de estos dos mecanismos no sirve para invocar al sistema operativo, puesto que el proceso se ejecuta a nivel de usuario mientras que el sistema operativo lo hace a nivel de núcleo y además ambos espacios de direcciones son distintos. Esto significa que los servicios del sistema operativo no se pueden solicitar mediante una instrucción de máquina CALL, o similar, es decir, mediante una llamada a procedimiento o función.

Por lo tanto, la activación del sistema operativo solamente se realiza mediante el mecanismo de las interrupciones. Cuando es un proceso en ejecución el que desea un servicio del sistema operativo utilizará una instrucción TRAP, que genera la interrupción pertinente. En los demás casos será una interrupción, interna o externa, la que reclame la atención del sistema operativo.

Repaso de las características del hardware de los sistemas informáticos

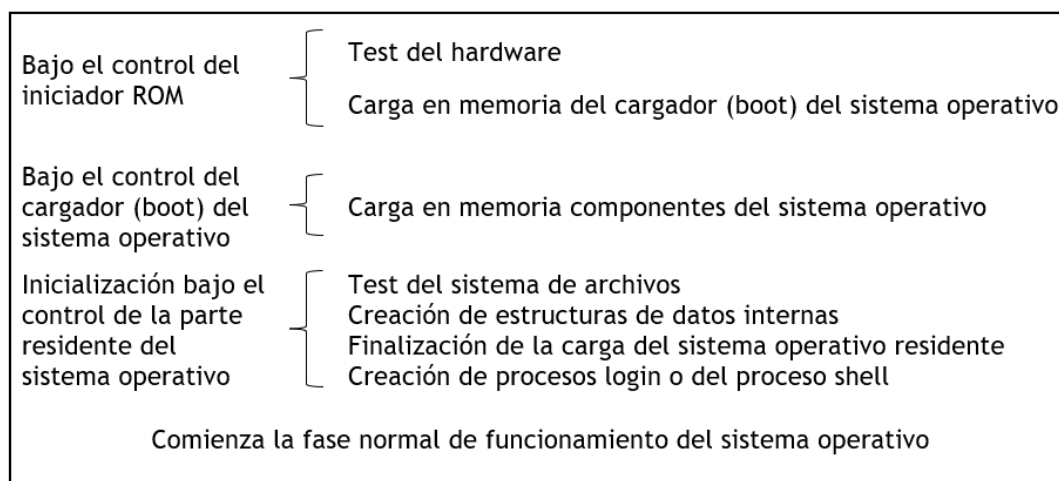
Un sistema informático de propósito general moderno está formado por una o más CPU's y un número de controladores de dispositivos conectados a través de un bus común que proporciona acceso entre los componentes y la memoria compartida. Cada controlador de dispositivos se encarga de un tipo de dispositivo específico. Por ejemplo, una unidad de disco, un dispositivo de audio o una pantalla gráfica.

Dependiendo del controlador, puede haber más de un dispositivo conectado. Por ejemplo, un puerto USB del sistema puede conectarse a un concentrador USB, al que pueden conectarse varios dispositivos. Un controlador de dispositivos mantiene un almacenamiento local en búfer y un conjunto de registros de propósito especial. El controlador de dispositivos es responsable de mover los datos entre los dispositivos periféricos que controla y su almacenamiento local en la memoria intermedia.

Normalmente, los sistemas operativos tienen un driver para cada controlador de dispositivo. Este driver entiende al controlador de dispositivo y proporciona al resto del sistema operativo una interfaz uniforme para el dispositivo. La CPU y los controladores de dispositivo pueden ejecutarse en paralelo, compitiendo por los ciclos de memoria. Para garantizar un acceso ordenado a la memoria compartida, un controlador de memoria sincroniza el acceso a la memoria. En los siguientes párrafos, describimos algunos aspectos básicos del funcionamiento de este sistema, centrándonos en algunos de sus aspectos clave.

Arranque de la computadora

Después de generar un sistema operativo, debe estar disponible para su uso por parte del hardware. Pero ¿cómo sabe el hardware dónde está el kernel o cómo cargar ese kernel? El proceso de arrancar las computadoras cargando el kernel se conoce como arranque del sistema o arranque de la computadora. En la mayoría de los sistemas, el proceso de arranque procede como sigue:



En esta sección, describiremos brevemente el proceso de arranque con más detalle.

Algunos sistemas de los computadores utilizan un proceso de arranque de varias etapas: Cuando los computadores se encienden por primera vez, se ejecuta un pequeño cargador de arranque ubicado en un firmware no volátil conocido como BIOS. Este cargador de arranque

inicial normalmente no hace más que cargar un segundo cargador de arranque, que se encuentra en una ubicación fija del disco, llamada bloque de arranque. El programa almacenado en el bloque de arranque puede ser lo suficientemente sofisticado como para cargar todo el sistema operativo en la memoria y comenzar su ejecución. Más típicamente, es un código simple (ya que debe caber en un solo bloque de disco) y sólo conoce la dirección en el disco y la longitud del resto del programa de arranque.

Muchos sistemas de computación recientes han sustituido el proceso de arranque basado en la BIOS por la UEFI (Unified Extensible Firmware Interface). UEFI tiene varias ventajas sobre BIOS, incluyendo un mejor soporte para sistemas de 64 bits y discos más grandes. Quizás la mayor ventaja es que UEFI es un gestor de arranque único y completo y, por lo tanto, es más rápido que el proceso de arranque de BIOS en varias etapas.

Tanto si se arranca desde BIOS como desde UEFI, el programa de arranque puede realizar diversas tareas. Además de cargar el archivo que contiene el programa del kernel en la memoria, también ejecuta diagnósticos para determinar el estado de la máquina, por ejemplo, inspeccionando la memoria y la CPU y descubriendo dispositivos. Si los diagnósticos pasan, el programa puede continuar con los pasos de arranque. El Bootstrap también puede inicializar todos los aspectos del sistema, desde los registros de la CPU hasta los controladores de dispositivos y el contenido de la memoria principal. Tarde o temprano, inicia el sistema operativo y monta el sistema de archivos raíz. Sólo en este momento se dice que el sistema está funcionando.

GRUB es un programa de arranque de código abierto para sistemas Linux y UNIX. Los parámetros de arranque del sistema se establecen en un archivo de configuración de GRUB, que se carga al inicio. GRUB es flexible y permite realizar cambios en el momento del arranque, incluyendo la modificación de los parámetros del kernel e incluso la selección entre diferentes kernels que pueden ser arrancados.

Cabe destacar que el mecanismo de arranque no es independiente del gestor de arranque. Por lo tanto, hay versiones específicas del cargador de arranque GRUB para BIOS y UEFI, y el firmware debe saber también qué cargador de arranque específico se va a utilizar.

El proceso de arranque de los sistemas móviles es ligeramente diferente al de los PC tradicionales. Por ejemplo, aunque su kernel está basado en Linux, Android no utiliza GRUB y, en su lugar, deja que los proveedores proporcionen cargadores de arranque. El cargador de arranque más común de Android es LK, por little kernel. Los sistemas Android utilizan la misma imagen comprimida del kernel que Linux, así como un sistema de archivos RAM inicial (initramfs).

Sin embargo, mientras que Linux descarta el initramfs una vez que se han cargado todos los controladores necesarios, Android mantiene el initramfs como sistema de archivos raíz del dispositivo. Una vez cargado el kernel y montado el sistema de archivos raíz, Android inicia el proceso de init y crea una serie de servicios antes de mostrar la pantalla de inicio.

Por último, los cargadores de arranque de la mayoría de los sistemas operativos, incluidos Windows, Linux y macOS, así como iOS y Android, permiten arrancar en modo de recuperación o en modo monousuario para diagnosticar problemas de hardware, reparar sistemas de archivos corruptos e incluso reinstalar el sistema operativo.

Elementos del procesador

Implícitamente hasta aquí asumimos una estructura general de un sistema informático típico, pero, un sistema informático puede organizarse de diferentes maneras, que podemos clasificar aproximadamente según el número de procesadores de propósito general utilizados.

Hace muchos años, la mayoría de los sistemas de computación utilizaban un único procesador (CPU) que contenía un único núcleo de procesamiento. El núcleo es el componente que ejecuta las instrucciones y los registros para almacenar datos localmente. La única CPU principal con su núcleo es capaz de ejecutar un conjunto de instrucciones de propósito general, incluyendo las instrucciones de los procesos. Estos sistemas tienen también otros procesadores de propósito especial. Pueden venir en forma de procesadores de dispositivos específicos, como los controladores de disco, teclado y gráficos.

Todos estos procesadores de propósito especial ejecutan un conjunto de instrucciones limitado y no ejecutan procesos. A veces, son gestionados por el sistema operativo, en el sentido de que éste les envía información sobre su próxima tarea y supervisa su estado. Por ejemplo, un microprocesador controlador de disco recibe una secuencia de peticiones del núcleo principal de la CPU e implementa su propia cola de disco y algoritmo de programación. Esta disposición libera a la CPU principal de la sobrecarga de la programación del disco. Los PC contienen un microprocesador en el teclado para convertir las pulsaciones en códigos que se envían a la CPU. En otros sistemas o circunstancias, los procesadores de propósito especial son componentes de bajo nivel incorporados al hardware. El sistema operativo no puede comunicarse con estos procesadores, sino que hacen su trabajo de forma autónoma. El uso de microprocesadores de propósito especial es común y no convierte un sistema de un solo procesador en un multiprocesador. Si sólo hay una CPU de propósito general con un único núcleo de procesamiento, el sistema es de un solo procesador. Sin embargo, según esta definición, muy pocos sistemas informáticos contemporáneos son sistemas de un solo procesador.

Recordemos entonces, antes de continuar, cuáles son las componentes tanto del procesador como del núcleo que acabamos de mencionar.

Algunas de las unidades que encontraremos dentro de un microprocesador actual, son las siguientes:

- Núcleo/s (Core) del procesador.
- Unidad de interfaz del bus o controlador del bus del sistema.
- Niveles compartidos de la memoria cache multinivel.
- Bus del microprocesador.
- Registros.
- En algunos diseños se incluye al controlador de memoria.
- Reloj

Los microprocesadores se caracterizan por la velocidad, la longitud de las palabras interna y externa, la arquitectura y el conjunto de instrucciones, que pueden ser fijas o microprogramadas. La combinación de estas características y no sólo el tiempo del ciclo, es

lo que determina el rendimiento de un procesador. La mayoría de los microprocesadores poseen un conjunto fijo de instrucciones. Los procesadores que son microprogramados tienen una memoria de control que contiene el microcódigo, o microinstrucción, definidor del conjunto de instrucciones del procesador.

Elementos del núcleo del procesador

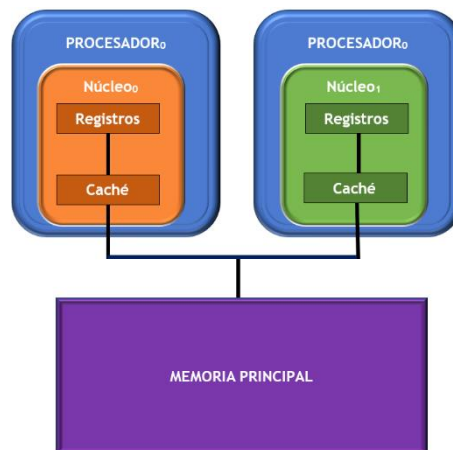
Habitualmente estará formado por los siguientes elementos:

- Unidad de control.
- Unidad aritmético - lógica.
- Unidad de coma flotante.
- Búfer de traducción anticipada de direcciones (TLB - Translation Lookaside Buffer).
- Unidad de administración de memoria (MMU - Memory Management Unit).
- Niveles exclusivos de la memoria cache multinivel.
- Contador de programa (Program Counter, PC) o contador ordinal o registro de próxima instrucción
- Registro de instrucción (Instruction Register, IR)
- Registro índice
- Registro de estado
- Registro de dirección de memoria (Memory Address Register, MAR)
- Registro intermedio de memoria (Memory Buffer Register, MBR) o registro de intercambio de memoria.
- Registros direccionables

En los computadores modernos, desde los dispositivos móviles hasta los servidores, los sistemas multiprocesadores dominan ahora el panorama de la computación. Tradicionalmente, estos sistemas tienen dos (o más) procesadores, cada uno con una CPU de un solo núcleo. Los procesadores comparten el bus del ordenador y, a veces, el reloj, la memoria y los dispositivos periféricos. La principal ventaja de los sistemas multiprocesador es el aumento del rendimiento. Es decir, al aumentar el número de procesadores, se espera hacer más trabajo en menos tiempo. Sin embargo, el aumento de velocidad con N procesadores no es N , sino que es menor que N . Cuando varios procesadores cooperan en una tarea, se produce una cierta sobrecarga para que todas las partes funcionen correctamente. Esta sobrecarga, más la contención por los recursos compartidos, reduce la ganancia esperada de los procesadores adicionales.

Los sistemas de multiprocesadores más comunes utilizan el multiprocesamiento simétrico (SMP), en el que cada procesador de la CPU homóloga realiza todas las tareas, incluidas las funciones del sistema operativo y los procesos de usuario. La figura siguiente ilustra una arquitectura SMP típica con dos procesadores, cada uno con su propia CPU. Obsérvese que cada procesador CPU tiene su propio conjunto de registros, así como una caché privada o local. Sin embargo, todos los procesadores comparten la memoria física a través del bus del sistema.

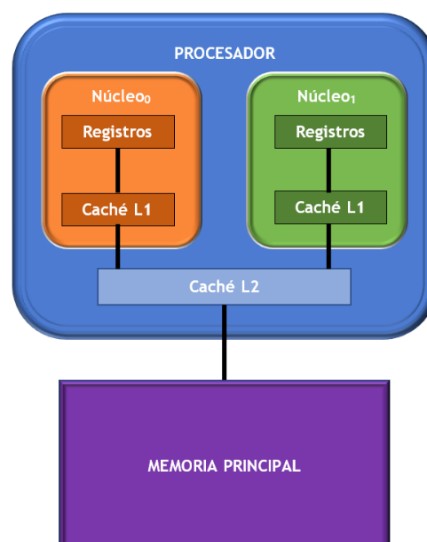
La ventaja de este modelo es que muchos procesos pueden ejecutarse simultáneamente, N procesos pueden ejecutarse si hay N CPU's, sin que el rendimiento se deteriore significativamente. Sin embargo, como las CPU's están separadas, una puede estar inactiva mientras otra está sobrecargada, lo que provoca ineficiencias. Estas ineficiencias pueden evitarse si los procesadores comparten ciertas estructuras de datos. Un sistema multiprocesador de este tipo permitirá que los procesos y recursos, como la memoria, se compartan dinámicamente entre los distintos procesadores y puede reducir la variación de la carga de trabajo entre los procesadores.



La definición de multiprocesador ha evolucionado con el tiempo y ahora incluye los sistemas multinúcleo, en los que varios núcleos de computación residen en un único chip. Los sistemas multinúcleo pueden ser más eficientes que los chips múltiples con un solo núcleo porque la comunicación en el chip es más rápida que la comunicación entre chips.

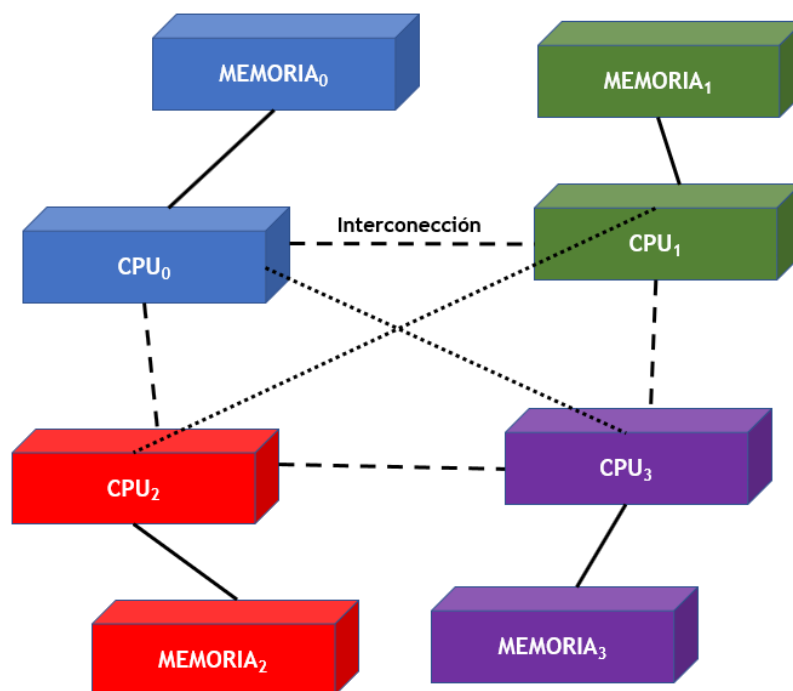
Además, un chip con varios núcleos consume mucha menos energía que varios chips de un solo núcleo, una cuestión importante para los dispositivos móviles y los computadores portátiles.

En la siguiente figura, podemos ver un diseño de doble núcleo con dos núcleos en el mismo chip procesador. En este diseño, cada núcleo tiene su propio conjunto de registros, así como su propia caché local, a menudo conocida como caché de nivel 1, o L1. También hay que tener en cuenta que la caché de nivel 2 (L2) es local en el chip, pero es compartida por los dos núcleos de procesamiento.



La mayoría de las arquitecturas adoptan este enfoque, combinando cachés locales y compartidas, donde las cachés locales de nivel inferior suelen ser más pequeñas y rápidas que las cachés compartidas de nivel superior. Aparte de las consideraciones arquitectónicas, como la caché, la memoria y la contención del bus, un procesador multinúcleo con N núcleos aparece ante el sistema operativo como N CPU's estándar. Esta característica presiona a los diseñadores de sistemas operativos, y a los programadores de aplicaciones, para que hagan un uso eficiente de estos núcleos de procesamiento. Prácticamente todos los sistemas operativos modernos, incluidos Windows, macOS y Linux, así como los sistemas móviles Android e iOS, admiten sistemas SMP multinúcleo.

Añadir más CPU's a un sistema multiprocesador aumentará la potencia de cálculo; sin embargo, como se ha sugerido antes, el concepto no se adapta muy bien, y una vez que añadimos demasiadas CPU's, la contención por el bus del sistema se convierte en un cuello de botella y el rendimiento empieza a degradarse. Un enfoque alternativo es proporcionar a cada CPU (o grupo de CPU's) su propia memoria local a la que se accede a través de un pequeño y rápido bus local. Las CPU's están conectadas por una interconexión de sistema compartida, de modo que todas las CPU's comparten un espacio de direcciones físico. Este enfoque, conocido como acceso no uniforme a la memoria, o NUMA, se ilustra en la siguiente figura. La ventaja es que, cuando una CPU accede a su memoria local, el acceso no sólo es rápido, sino que además no hay contención en la interconexión del sistema.



Así, los sistemas NUMA pueden escalar más eficazmente a medida que se añaden más procesadores. Una desventaja potencial de un sistema NUMA es el aumento de la latencia cuando una CPU debe acceder a la memoria remota a través de la interconexión del sistema, lo que crea una posible penalización del rendimiento. En otras palabras, por ejemplo, la CPU₀ no puede acceder a la memoria local de la CPU₃ con la misma rapidez con la que puede acceder a su propia memoria local, reduciendo el rendimiento. Los sistemas operativos pueden minimizar esta penalización de NUMA a través de la programación cuidadosa de la CPU y la gestión de la memoria. Dado que los sistemas NUMA pueden escalar para acomodar

un gran número de procesadores, se están haciendo cada vez más populares en los servidores, así como en los sistemas de computación de alto rendimiento.

Por último, los servidores Blade son sistemas en los que se colocan múltiples placas de procesador, placas de E/S y placas de red en el mismo chasis. La diferencia con los sistemas multiprocesador tradicionales es que cada placa de procesador Blade arranca de forma independiente y ejecuta su propio sistema operativo. Algunas placas de servidores Blade son también multiprocesadores, lo que difumina las fronteras entre los computadores. En esencia, estos servidores consisten en múltiples sistemas multiprocesadores independientes.

Otro tipo de sistema multiprocesador es el sistema en clúster, que reúne múltiples CPU's. Los sistemas en clúster se diferencian de los sistemas multiprocesadores descritos en los párrafos anteriores en que están compuestos por dos o más sistemas individuales, o nodos, unidos entre sí; cada nodo suele ser un sistema multinúcleo. Estos sistemas se consideran poco acoplados. Hay que tener en cuenta que la definición de clúster no es concreta; muchos paquetes comerciales y de código abierto se esfuerzan por definir qué es un clúster y por qué una forma es mejor que otra. La definición generalmente aceptada es que los clusters comparten el almacenamiento y están estrechamente vinculados a través de una red de área local LAN o de una interconexión más rápida, como InfiniBand.

El clustering se utiliza normalmente para proporcionar un servicio de alta disponibilidad, es decir, un servicio que continuará incluso si uno o más sistemas del clúster fallan. Generalmente, se obtiene una alta disponibilidad añadiendo un nivel de redundancia en el sistema. En los nodos del clúster se ejecuta una capa de software de clúster. Cada nodo puede monitorizar uno o varios de los otros (a través de la red). Si la máquina monitorizada falla, la máquina de monitorización puede hacerse cargo de su almacenamiento y reiniciar las aplicaciones que se estaban ejecutando en la máquina que ha fallado. Los usuarios y clientes de las aplicaciones sólo ven una breve interrupción del servicio.

La alta disponibilidad proporciona una mayor fiabilidad, que es crucial en muchas aplicaciones. La capacidad de seguir proporcionando un servicio proporcional al nivel del hardware superviviente se denomina degradación elegante. Algunos sistemas van más allá de la degradación elegante y se denominan tolerantes a fallos, porque pueden sufrir un fallo de cualquier componente individual y seguir funcionando. La tolerancia a los fallos requiere un mecanismo que permita detectar, diagnosticar y, si es posible, corregir el fallo.

El clustering puede estructurarse de forma asimétrica o simétrica. En su forma asimétrica, una máquina está en modo de espera en caliente mientras la otra ejecuta las aplicaciones. La máquina en espera en caliente no hace nada más que supervisar al servidor activo. Si ese servidor falla, el host en espera en caliente se convierte en el servidor activo. En el caso de un clustering simétrico, dos o más hosts ejecutan aplicaciones y se monitorizan mutuamente. Esta estructura es obviamente más eficiente, ya que utiliza todo el hardware disponible. Sin embargo, requiere que más de una aplicación esté disponible para ejecutarse.

Dado que un clúster está formado por varios sistemas informáticos conectados a través de una red, los clusters también pueden utilizarse para proporcionar entornos informáticos de alto rendimiento. Estos sistemas pueden proporcionar una potencia de cálculo significativamente mayor que los sistemas de un solo procesador o incluso los sistemas SMP, ya que pueden ejecutar una aplicación de forma simultánea en todos los computadores del

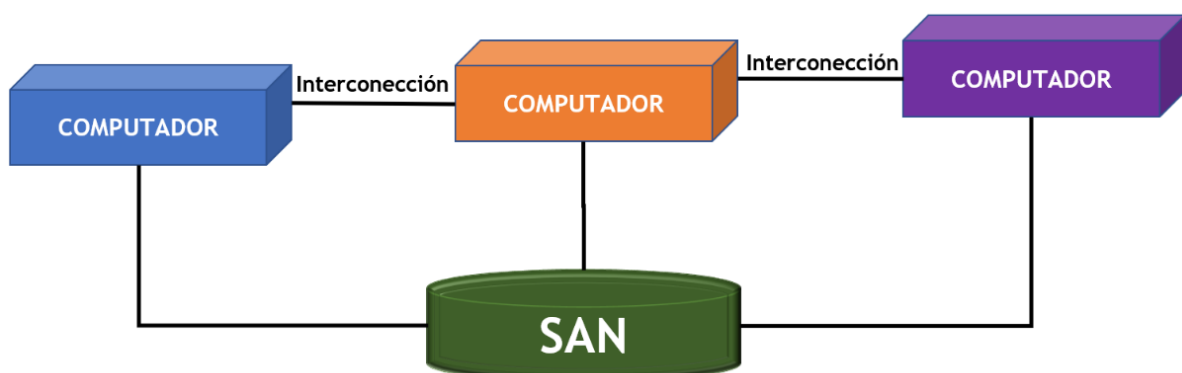
clúster. Sin embargo, la aplicación debe haber sido escrita específicamente para aprovechar el clúster.

Esto implica una técnica conocida como paralelización, que divide un programa en componentes separados que se ejecutan en paralelo en núcleos individuales de un computador o los computadores de un clúster. Normalmente, estas aplicaciones se diseñan de forma que, una vez que cada nodo informático del clúster ha resuelto su parte del problema, los resultados de todos los nodos se combinan en una solución final.

Otras formas de clusters son los clusters paralelos y los clusters sobre una red de área amplia (WAN). Los clusters paralelos permiten que varios hosts accedan a los mismos datos en un almacenamiento compartido. Dado que la mayoría de los sistemas operativos carecen de soporte para el acceso simultáneo a los datos por parte de varios hosts, los clusters paralelos suelen requerir el uso de versiones especiales de software y versiones especiales de aplicaciones.

Por ejemplo, Oracle Real Application Cluster es una versión de la base de datos de Oracle que ha sido diseñada para funcionar en un clúster paralelo. Cada máquina ejecuta Oracle y una capa de software controla el acceso al disco compartido. Cada máquina tiene acceso completo a todos los datos de la base de datos. Para proporcionar este acceso compartido, el sistema también debe proporcionar control de acceso y bloqueo para garantizar que no se produzcan operaciones conflictivas. Esta función, comúnmente conocida como gestor de bloqueo distribuido (DLM), se incluye en algunas tecnologías de clúster.

Algunos productos de clúster soportan miles de sistemas en un clúster, así como nodos agrupados que están separados por kilómetros. Muchas de estas mejoras son posibles gracias a las redes de área de almacenamiento (SAN: Storage Area Network), que permiten que muchos sistemas se conecten a un grupo de almacenamiento. Si las aplicaciones y sus datos se almacenan en la SAN, el software del clúster puede asignar la aplicación para que se ejecute en cualquier host que esté conectado a la SAN. Si el host falla, cualquier otro host puede tomar el relevo. En un clúster de bases de datos, docenas de hosts pueden compartir la misma base de datos, lo que aumenta enormemente el rendimiento y la fiabilidad. La figura siguiente muestra la estructura general de un sistema en clúster.



Ciclo de instrucción

La función básica que realiza una computadora es la ejecución de programas. El programa por ejecutar está compuesto por un conjunto de instrucciones almacenadas en memoria. La Unidad Central de Procesamiento (CPU: Central Processing Unit) es precisamente la que se

encarga de ejecutar las instrucciones especificadas en el programa. Para adquirir una mejor comprensión de esta función y de la forma en la que los principales componentes de la computadora interactúan para ejecutar un programa, es preciso considerar con detalle el proceso de ejecución del programa.

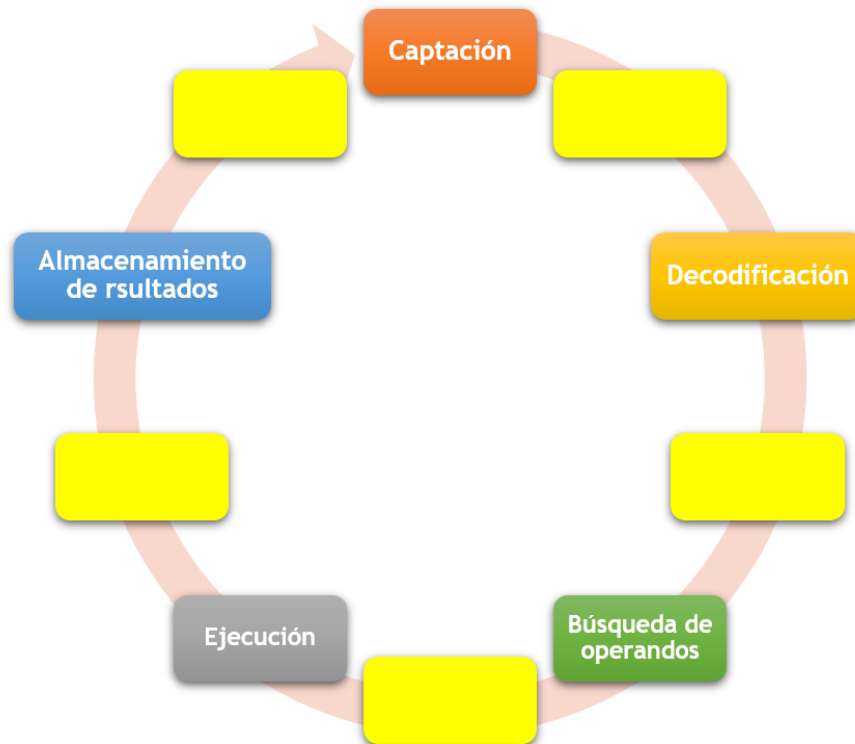
El punto de vista más sencillo considera el procesamiento de una instrucción constituido por dos etapas, la CPU lee (capta: fetch en inglés) la instrucción de memoria, y la ejecuta. La ejecución del programa consiste en la repetición del proceso de captación de instrucción y ejecución de instrucción. Por supuesto, la ejecución de la instrucción puede a su vez estar compuesta de cierto número de pasos. Por el momento, se puede justificar la separación del procesamiento de una instrucción en las etapas de captación y ejecución.

La captación de una instrucción es una operación común a todas las instrucciones, y consiste en la lectura de la instrucción de una posición de memoria. La ejecución de la instrucción puede suponer varias operaciones y depende de la naturaleza de la instrucción. La ejecución del programa se para sólo si la máquina se desconecta, se produce algún tipo de error, o se encuentra una instrucción del programa que detiene a la computadora.

El procesamiento que requiere una instrucción se denomina ciclo de instrucción. Los dos subciclos se denominan ciclo de búsqueda y ciclo de ejecución. En conjunto estos subciclos contienen un conjunto de pasos, etapas o fases, cuyo número puede variar dependiendo del diseño desarrollado. Un concepto que, como veremos oportunamente, resulta especialmente adecuado para la segmentación de instrucciones. Por ejemplo, podríamos tener que el ciclo de instrucción está compuesto por las siguientes etapas:

- Actualización o Incremento del contador de programa y Captación de la instrucción. La actualización del contador de programa es el primer paso, totalmente automático, dirigido a la obtención de la siguiente instrucción a ejecutarse. Normalmente, esto implica añadir un número fijo a la dirección de la instrucción previa. Por ejemplo, si las instrucciones tienen un tamaño de 16 bits y la memoria se organiza en palabras de 16 bits, se suma 1 a la dirección previa. En cambio, si la memoria se organiza en bytes (8 bits) direccionables individualmente, entonces hay que sumar 2 a la dirección previa. A continuación de la actualización la CPU lee la instrucción desde su posición en memoria.
- Decodificación de la operación indicada en la instrucción. El decodificador de la CPU analiza la instrucción para determinar el tipo de operación a realizar y el (los) operando(s) a utilizar.
- Búsqueda de operandos. Está formada por el Cálculo de la dirección del operando (CDO) y por la Captación de operando (CO). Si la instrucción implica una referencia a un operando en memoria o disponible mediante E/S, determina la dirección del operando. Luego de determinar la dirección del operando la CPU capta al operando desde memoria o se lee desde el dispositivo de E/S
- Ejecución de la Operación con los operandos. Realiza la operación indicada en la instrucción.
- Cálculo de la dirección del resultado y Almacenamiento del resultado. Si la instrucción implica una referencia a un resultado que se guardará en memoria interna o externa,

determina la dirección del resultado. Posteriormente se escribe el resultado en memoria o lo saca a través de un dispositivo de E/S.



En la figura precedente, los bloques amarillos entre cada etapa representan cada uno de los ciclos de máquina, el cual se define como el tiempo requerido para pasar de una etapa a la siguiente, cuya duración viene determinada por la duración de la etapa más lenta.

Al comienzo de cada ciclo de instrucción, la CPU capta una instrucción de memoria. En una CPU típica, se utiliza un registro llamado contador de programa (CP) para seguir la pista de la instrucción que debe captarse a continuación. A no ser que se indique otra cosa, la CPU siempre incrementa el CP después de captar cada instrucción de forma que captará la siguiente instrucción de la secuencia (es decir, la instrucción situada en la siguiente dirección de memoria). Considérese, por ejemplo, una computadora en la que cada instrucción ocupa una palabra de memoria de 16 bits. Se supone que el contador de programa almacena el valor 300. La CPU captará la próxima instrucción almacenada en la posición 300. En los siguientes ciclos de instrucción, captará las instrucciones almacenadas en las posiciones 301, 302, 303, y así sucesivamente. La instrucción captada se almacena en un registro de la CPU conocido como registro de instrucción (RI). La instrucción se escribe utilizando un código binario que especifica la acción que debe realizar la CPU. La CPU interpreta la instrucción y lleva a cabo la acción requerida. En general, estas acciones se dividen en cuatro categorías:

- Procesador-memoria. Se pueden transferir datos desde el procesador a la memoria o viceversa.
- Procesador-E/S. Se pueden enviar datos a un dispositivo periférico o recibirlos desde el mismo, transfiriéndolos entre el procesador y un módulo de E/S.
- Procesamiento de datos. El procesador puede realizar algunas operaciones aritméticas o lógicas sobre los datos.

- Control. Una instrucción puede especificar que se va a alterar la secuencia de ejecución. Por ejemplo, el procesador puede leer una instrucción de la posición 149, que especifica que la siguiente instrucción estará en la posición 182. El procesador almacenará en el contador del programa un valor de 182. Como consecuencia, en la siguiente fase de búsqueda, se leerá la instrucción de la posición 182 en vez de la 150.

Una ejecución de una instrucción puede involucrar una combinación de estas acciones.

Interrupciones

Una interrupción es un pulso eléctrico generado por el hardware conectado al bus del computador, o por un proceso, que provoca que la CPU no continúe ejecutando temporariamente las instrucciones del proceso. De esta forma, el sistema de interrupciones consiste en una serie de bifurcaciones externas al programa en ejecución, provocadas por señales que provienen del exterior o del interior de la CPU. Esto ocurre así porque al interrumpirse la ejecución de un programa en forma temporaria o definitiva, para que sea atendido algún otro evento, toma el control de la CPU el sistema operativo para ocuparse de atender la interrupción. Luego, un sistema de interrupciones es aquel que fuerza el cambio de estado de la CPU permitiendo que varios programas se alternen en el uso de esta. Estos pulsos eléctricos están destinados a:

- Indicar culminación de períodos de reloj
- Permitir acciones de entradas/salidas
- Uso de DMA
- Uso de Cachés
- La ejecución de un trap (trampa) genera una interrupción y puede ser por un error o solicitud del usuario.

Aunque el quiebre de la secuencia normal de ejecución de un programa al introducir una bifurcación es un aspecto muy importante de las interrupciones, también existe otro aspecto importante y es que, generalmente, deben permitir que el programa que estaba siendo ejecutado en el momento de ocurrir la interrupción pueda seguir siendo ejecutado en las condiciones en las que estaba en el instante de la interrupción.

Para ello es imprescindible que se salve el estado de la computadora en el momento de la interrupción, para poder restaurarlo al devolver el control al programa interrumpido. Por el contrario, si el programa interrumpido no puede continuarse, el despachador del sistema operativo indicará cuál es el siguiente programa que usará la CPU. Una ventaja importante del sistema interrupciones es que es una forma económica de llamar la atención de la CPU, de modo que la CPU no necesita escrutar constantemente los dispositivos para ver si alguno requiere atención. Hoy en día, prácticamente todas las computadoras disponen de un mecanismo mediante el que otros módulos (E/S, memoria) pueden interrumpir el procesamiento normal de la CPU.

Ahora bien, la computadora tiene, además de la memoria principal, una serie de elementos de memoria que se van modificando de forma dinámica. Para que el programa interrumpido pueda seguir funcionando correctamente, estos elementos deberán volver a tomar el mismo

valor que tenían cuando el programa fue interrumpido. Además, la zona de memoria principal asignada al programa interrumpido deberá haber sido respetada.

Llamaremos **estado de la computadora** o **estado del procesador** al contenido de estos elementos de memoria interna, necesarios para que un programa pueda seguir funcionando correctamente. Dependiendo de lo compleja que sea la computadora, su estado estará definido por más o menos elementos de memoria, pero en forma general se pueden destacar los siguientes:

- Contador de programa.
- Registros generales o banco de registros.
- Registros aritméticos auxiliares, siempre que su valor pueda tener sentido entre instrucciones sucesivas.
- Bits de estado aritméticos, como los bits de acarreo, desbordamiento, cero o negativo.
- Bits de entrada / salida, como los bits de inhibición de interrupción, de máscara o de nivel.
- Bits o registros de modificación del mapa de memoria, como los bits o los registros de ampliación del mapa de memoria.
- Registros de clave o de protección de memoria, que limitan la zona de la memoria principal accesible para un usuario.
- Bits de modo de ejecución de la computadora.
- Máscara de interrupción o bits de interrupciones habilitadas / inhabilitadas

Con parte de este conjunto de información algunos diseños de procesadores presentan el concepto de **palabra de estado del programa** o **PSW (Program Status Word)** que está conformado por los bits de estado aritméticos, bits indicadores de interrupción y los bits indicadores del modo de ejecución.

Los **modos** en los que puede estar una computadora son: activo, en espera, supervisor, problema, enmascarado y desenmascarado.

- Modo activo: La CPU está ejecutando una instrucción.
- Modo en espera: Significa que la CPU no está ejecutando instrucción alguna.
- Modo supervisor: El procesador puede ejecutar la totalidad de las instrucciones disponibles en el lenguaje de máquina, ya sean privilegiadas o no privilegiadas.
- Modo problema: La CPU tiene sólo permitido el uso de las instrucciones no privilegiadas.
- Modo enmascarado: La CPU está impedida de atender cualquier solicitud de interrupción.
- Modo desenmascarado: La CPU atiende inmediatamente cualquier solicitud de interrupción.

El **enmascaramiento** prohíbe que se atiendan las interrupciones en el momento en que son solicitadas. En algunos casos el hardware memoriza la solicitud y en otros casos esta se pierde, en este caso la solicitud deberá hacerse nuevamente. En cada momento la CPU puede estar enmascarada contra todas, algunas o ninguna clase de interrupción. Hay una parte

mínima de este estado que la tiene que salvar la CPU al aceptar la interrupción, puesto que reside en los registros que son modificados por la propia interrupción.

Ahora podemos establecer que la CPU ha de salvar:

- El contador de programa, puesto que recibe la dirección de la rutina de tratamiento de la interrupción.
- El registro de estado (PSW), que, entre otras cosas, incluye la información de enmascaramiento de las interrupciones, puesto que esta información ha de ser modificada para evitar que se acepten interrupciones menos prioritarias hasta que, en su caso, la rutina de tratamiento de la interrupción decida que se puede.

El resto de la información de estado, como los registros de uso general y los otros bits, ha de ser salvado por la rutina de tratamiento de la interrupción. Esto es lo primero que debe hacer esta rutina antes de modificar alguno de estos elementos. Finalmente, antes de volver a dar el control al programa interrumpido hay que restaurar todos los registros salvados por la rutina de tratamiento de la interrupción y hay que dejar la pila exactamente igual a como estaba antes de que se atendiera la interrupción.

Las computadoras suelen disponer de una instrucción especial de retorno de interrupción (RETI), que permite devolver el control al programa interrumpido, haciendo las operaciones inversas del ciclo de reconocimiento de la interrupción, esto es, restaurando el registro de estado y el contador del programa interrumpido.

De acuerdo con el funcionamiento de la CPU, ésta va ejecutando, de forma inamovible, las instrucciones sucesivas y contiguas de un programa. Sólo se quiebra esta secuencia cuando se encuentra una bifurcación, que consiste en pasar a ejecutar una instrucción diferente de la consecutiva en memoria. Ahora bien, como ya sabemos, existen situaciones en las que puede resultar interesante suspender la ejecución de un programa y pasar a ejecutar otro. Esto ocurre, por ejemplo, en los siguientes casos:

- Cuando aparece un error de ejecución, tal como desbordamiento en el resultado de una operación aritmética.
- Cuando aparece un error de paridad en una lectura en memoria.
- Al intentar ejecutar una instrucción con código de operación prohibido o inexistente.
- Al intentar acceder a una zona protegida de la memoria.
- Cuando un dispositivo requiere la atención de la CPU.

En estos y otros casos debe producirse una ruptura de secuencia no programada, puesto que no se produce explícitamente por una instrucción de bifurcación del programa en ejecución. El mecanismo para producir estas rupturas de secuencia no programadas es similar al empleado para las bifurcaciones condicionales. En efecto, todas las condiciones que pueden producir esta ruptura de secuencia automática se reflejan en los correspondientes biestables de estado. Además, todas las instrucciones llevan al final de su ciclo una bifurcación condicional implícita sobre el valor de estos biestables. De esta forma, en cuanto uno de ellos se activa, se suspende la ejecución del programa en curso y se bifurca.

Cuando la causa de esta bifurcación no programada proviene del exterior de la CPU, se dice que se ha producido una **interrupción externa**. Por ejemplo, una unidad de disco puede

interrumpir a la CPU cuando termina una operación de entrada o de salida. Otros ejemplos son: cuando se pulsa la tecla de interrupción del teclado, cuando llega desde otra CPU una señal en un sistema de multiprocesamiento, cuando se reciben señales emitidas por dispositivos robóticos usados en procesos industriales. Por el contrario, cuando la causa de la bifurcación no programada es interna a la CPU, es decir que es iniciada específicamente por el proceso en ejecución, se la suele llamar **interrupción interna** o **trampa** o **cepo** o **trap** o **excepción**. La diferencia fundamental entre ambas es que las internas son sincrónicas con el proceso y las externas son asincrónicas porque son causadas por algún evento que está relacionado o no con el proceso en ejecución.

Existen dos formas para determinar cómo generar la dirección de bifurcación. La primera de ellas consistía en bifurcar a una posición prefijada o precableada, y es la forma más frecuente en el caso de los cepos. La otra técnica es empleada para la atención de interrupciones externas y es el dispositivo quién proporciona la dirección de bifurcación. En general, la dirección de bifurcación corresponde a un programa del sistema operativo que trata a la interrupción interna o externa y, luego, restituye el estado y devuelve el control al proceso interrumpido.

Ciertos sistemas informáticos tienen dos tipos de eventos que hacen que el procesador suspenda la ejecución del flujo de instrucciones en curso y responda al evento: interrupciones por hardware e interrupciones por software.

Las **interrupciones por hardware**, que se generan por una señal de hardware y pueden ocurrir en momentos aleatorios durante la ejecución de un proceso.

En este esquema existen dos clases de interrupciones por hardware: **enmascarables** y **no enmascarables**. Las primeras son aquellas en las que el procesador decide si quiere o no ser interrumpido, mientras que las segundas siempre interrumpen al proceso en ejecución.

Cuando se habla de una interrupción enmascarable significa que, bajo control del software, el procesador puede aceptar o ignorar (enmascarar) la señal de interrupción. Para ello se envía una señal a la conexión INTR, y el procesador la atiende o la ignora en función del contenido de un bit (IF) en un registro (FLAGS) que puede estar habilitado o deshabilitado. En este caso, cuando se recibe la señal, el procesador concluye la instrucción que estuviese en proceso y a continuación responde con una combinación de señales en algunas de sus conexiones componiendo una sucesión de dos señales INTA (Interrupt Acknowledge). La primera señal es simplemente un aviso; la segunda es una petición para que quién interrumpió coloque en el bus de datos un byte con el número de interrupción, de forma que el procesador pueda localizar el servicio solicitado. El valor recibido (0-255) es multiplicado por 4 para calcular la dirección del vector correspondiente en la tabla de vectores de interrupción, lo que se realiza mediante un desplazamiento binario de dos posiciones a la izquierda.

Que la interrupción es no enmascarable significa que la misma no puede ser deshabilitada por software. Este tipo de interrupciones ocurren cuando se recibe una señal en la conexión NMI (Nonmaskable Interrupt) del procesador. Se reservan para casos en que es crítica la respuesta, por ejemplo, que se detecte un error de paridad en la memoria. Además, son de prioridad más alta que las enmascarables.

Entre las interrupciones por hardware podemos mencionar a las interrupciones por error de máquina, por entrada / salida, por reinicio del sistema.

Las **interrupciones por error de máquina** son ocasionadas por el mal funcionamiento del hardware. Es una interrupción asincrónica, externa y el sistema operativo puede inhibirse de atender algunas de ellas. Por ejemplo: Falla de circuitos en la CPU; Falla de circuitos en la memoria central.

Las **interrupciones por entrada / salida** son generadas por el canal procesador encargado de la realización de la operación de entrada / salida. Estas interrupciones le indican a la CPU el cambio del estado del canal o de un dispositivo. Se producen cuando finaliza una operación de entrada / salida o cuando un dispositivo pasa al estado listo. Son externas y asincrónicas.

La **interrupción de reinicio** ocurre cuando se presiona el botón de reinicio de la consola o cuando desde otro procesador llega una instrucción de reinicio en un sistema de multiprocesamiento. Es externa y asincrónica.

Por lo general la interrupción es atendida cuando termina la ejecución de la instrucción que se está procesando o cuando la interrupción es detectada. Por ejemplo, tenemos que el fin de una entrada / salida es un tipo de interrupción que puede ser atendida cuando el procesador termine con la ejecución de una instrucción. Otras como error de programa o error de máquina deben ser atendidas no bien se producen.

Las **excepciones o interrupciones por software** son generadas desde el software y provocadas por la ejecución de una instrucción. Las hay de 2 clases diferentes: las detectadas por el procesador y las programadas. Las primeras se producen cuando el procesador encuentra un error mientras intenta ejecutar una instrucción. Las segundas son producidas por instrucciones que generan una interrupción generada por la propia CPU. Dentro de esta clase de interrupciones encontramos a las interrupciones por llamada al supervisor (Sistema Operativo) y de verificación de programa.

Las **interrupciones por llamada al supervisor (SVC)** son iniciadas por un proceso en ejecución que ejecute una instrucción de llamada al supervisor (SVC). Una SVC es una petición generada por el usuario de un servicio particular del sistema, como realizar una operación de entrada / salida, obtener más memoria o comunicarse con el operador del sistema. El mecanismo de las SVC ayuda a proteger el sistema operativo de las acciones de los usuarios. Un usuario no puede entrar arbitrariamente al sistema operativo, sino que debe solicitarle un permiso por intermedio de una SVC. El sistema operativo está al tanto de todos los usuarios que intentan rebasar sus límites y puede rechazar ciertas peticiones si el usuario no tiene los privilegios necesarios. Cuando ocurre este tipo de interrupción la CPU debe atenderla inmediatamente. Son internas y sincrónicas.

Como ejemplo de ellas se pueden mencionar: Cumplir funciones sobre periféricos; Líneas de código como write, cout, cin, scanf, printf, read; Modificar el PCB de un proceso; Cumplir funciones referentes a procesos (solicitar su ejecución, modificar su prioridad, cancelarlo); Comunicarse con el operador; Cumplir funciones sobre la memoria (liberar memoria virtual o real, liberar buffers).

Las **interrupciones de verificación de programa** son causadas por una amplia clase de problemas que pueden ocurrir cuando se ejecutan las instrucciones de lenguaje de máquina de un programa. Dichos problemas incluyen la división por cero, el exceso o defecto de los números que pueden ser manejados por las operaciones aritméticas, la presencia de datos (con los cuales se está trabajando) con un formato erróneo, el intento de ejecutar un código

de operación inválido, el intento de hacer referencia a un lugar de la memoria que esté fuera de los límites de la memoria real, el intento por parte de un proceso de usuario de ejecutar una instrucción privilegiada, el intento de hacer referencia a un recurso protegido y por falta de espacio en memoria real. Es una interrupción sincrónica e interna, y el sistema operativo puede inhibirse de atender algunas de ellas. Muchos sistemas ofrecen a los usuarios la opción de especificar las rutinas que deben ejecutarse cuando ocurra una interrupción de este tipo.

Tanto en el caso de una interrupción por hardware como en una por software, el procesador guarda en una pila el contexto del proceso actual y pasa a ejecutar una rutina predefinida del sistema operativo para atender el evento.

Desde el punto de vista del programa de usuario, una interrupción es precisamente eso: una interrupción en la secuencia normal de funcionamiento. Cuando el procesamiento de la interrupción se completa, la ejecución prosigue. Así, el programa de usuario no tiene que incluir ningún código especial para posibilitar las interrupciones; el procesador y el sistema operativo son los responsables de detener el programa de usuario y después permitir que prosiga en el mismo punto.

Para permitir el uso de interrupciones, se añade un **ciclo de interrupción** al final de cada ciclo de instrucción. En el ciclo de interrupción, el procesador comprueba si se ha producido cualquier interrupción, hecho indicado por la presencia de una señal de interrupción. Si no hay interrupciones pendientes, el procesador continúa con la etapa de captación de la siguiente instrucción y lee la siguiente instrucción del programa actual. Si está pendiente una interrupción, el procesador suspende la ejecución del programa actual y ejecuta la rutina del manejador de interrupción. La rutina del manejador de interrupción es generalmente parte del sistema operativo.

Este ciclo de interrupción estará dividido en otros dos ciclos llamados: **ciclo de concesión de interrupción** y **ciclo de tratamiento de la interrupción**, debemos diferenciarlos claramente. El ciclo de concesión de interrupción consiste en hacer que la CPU realice una bifurcación no programada, mientras que, en el ciclo de tratamiento de la interrupción, el programa o rutina de tratamiento de la interrupción deberá realizar las operaciones necesarias para atender al dispositivo que solicitó la interrupción.

Puesto que la interrupción es una bifurcación, hay que establecer un mecanismo para determinar la dirección de bifurcación, dirección donde se encuentra su rutina de tratamiento. Existen varias técnicas para definir esta dirección durante el ciclo de concesión de interrupción siendo la más utilizada la del **direccionamiento relativo indirecto**. En este caso, la dirección que envía el dispositivo es la posición relativa de una tabla o vector, residente en memoria principal, que contiene las direcciones iniciales de las rutinas de tratamiento de interrupciones. Mediante un direccionamiento indirecto, se accede a la entrada correspondiente del vector y se bifurca a la rutina deseada. Esta técnica se conoce con el nombre de interrupciones vectorizadas o interrupciones con vectores completos o vectores de interrupción.

Consideremos ahora con más detalle el papel del procesador en una interrupción, por ejemplo, de E/S. Cuando se produce una interrupción se disparan una serie de eventos en el procesador, tanto a nivel hardware como software. Entonces, cuando un dispositivo de E/S termina una operación de E/S, se produce la siguiente secuencia de eventos en el hardware:

- 1.- El dispositivo envía una señal de interrupción al procesador.
- 2.- El procesador termina la ejecución de la instrucción en curso antes de responder a la interrupción, como se indicó en el ciclo de instrucción.
- 3.- El procesador comprueba si hay interrupciones, determina que hay una, y envía una señal de reconocimiento al dispositivo que originó la interrupción. La señal de reconocimiento hace que el dispositivo desactive su señal de interrupción.
- 4.- Ahora el procesador necesita prepararse para transferir el control a la rutina de interrupción. Para empezar, debe guardar la información necesaria para continuar el proceso en curso en el punto en que se interrumpió. La información mínima que se precisa es el estado del procesador, que es una estructura lógica llamada palabra de estado del programa, y la posición de la siguiente instrucción a ejecutar, que está contenida en el contador de programa. Esta información se puede introducir en la pila de control del sistema.
- 5.- Después, el procesador carga el contador de programa con la posición de inicio del programa de gestión de la interrupción solicitada. Según sea la arquitectura de la computadora y el diseño del sistema operativo, puede haber un sólo programa, uno por cada tipo de interrupción, o uno por cada dispositivo y cada tipo de interrupción. Si hay más de una rutina de gestión de interrupción, el procesador debe determinar qué programa llamar. Esta información puede haber sido incluida en la señal de interrupción original, o el procesador puede tener que enviar una solicitud al dispositivo que originó la interrupción para que éste responda con la información que se precise.
- 6.- Una vez que el contador de programa se ha cargado, el procesador continúa con el ciclo de instrucción siguiente, que empieza con la captación de instrucción. Puesto que la instrucción a captar viene determinada por el contenido del contador de programa, el control se transfiere al programa de gestión de interrupción.
- 7.- Hasta este momento, se han almacenado en la pila del sistema el contador de programa y el PSW del programa interrumpido. Sin embargo, hay otra información que se considera parte del estado de un programa en ejecución. En concreto, se deben guardar los contenidos de los registros del procesador puesto que estos registros pueden ser utilizados por la rutina de interrupción. Usualmente, la rutina de gestión de interrupción empezará almacenando en la pila los contenidos de todos los registros.
- 8.- La rutina de gestión de la interrupción puede continuar ahora procesando la interrupción. Esto incluirá el examen de la información de estado relativa a la operación de E/S o a cualquier otro evento que causará la interrupción. También puede implicar el envío al dispositivo de E/S de órdenes o señales de reconocimiento adicionales.
- 9.- Cuando el procesamiento de la interrupción ha terminado, los valores de los registros almacenados se recuperan de la pila y se vuelven a almacenar en los registros.
- 10.- El paso final es recuperar los valores del PSW y del contador de programa desde la pila. Como resultado, la siguiente instrucción que se ejecute pertenecerá al programa previamente interrumpido.

Normalmente, el sistema operativo determina el origen de la interrupción y realiza todas las acciones que sean necesarias. Por ejemplo, en el caso que hemos analizado, el gestor

determina qué módulo de E/S generó la interrupción, y puede saltar a un programa que escribe más datos en ese módulo de E/S. Cuando la rutina de gestión de interrupción se completa, el procesador puede proseguir la ejecución del programa de usuario en el punto en el que se interrumpió. Es claro que este proceso supone una cierta penalización (overhead). Deben ejecutarse instrucciones extra (en el gestor de interrupción) para determinar el origen de la interrupción y para decidir la acción apropiada. No obstante, debido a la relativamente gran cantidad de tiempo que se perdería simplemente por la espera asociada a la operación de E/S, el procesador puede emplearse de manera mucho más eficiente utilizando interrupciones.

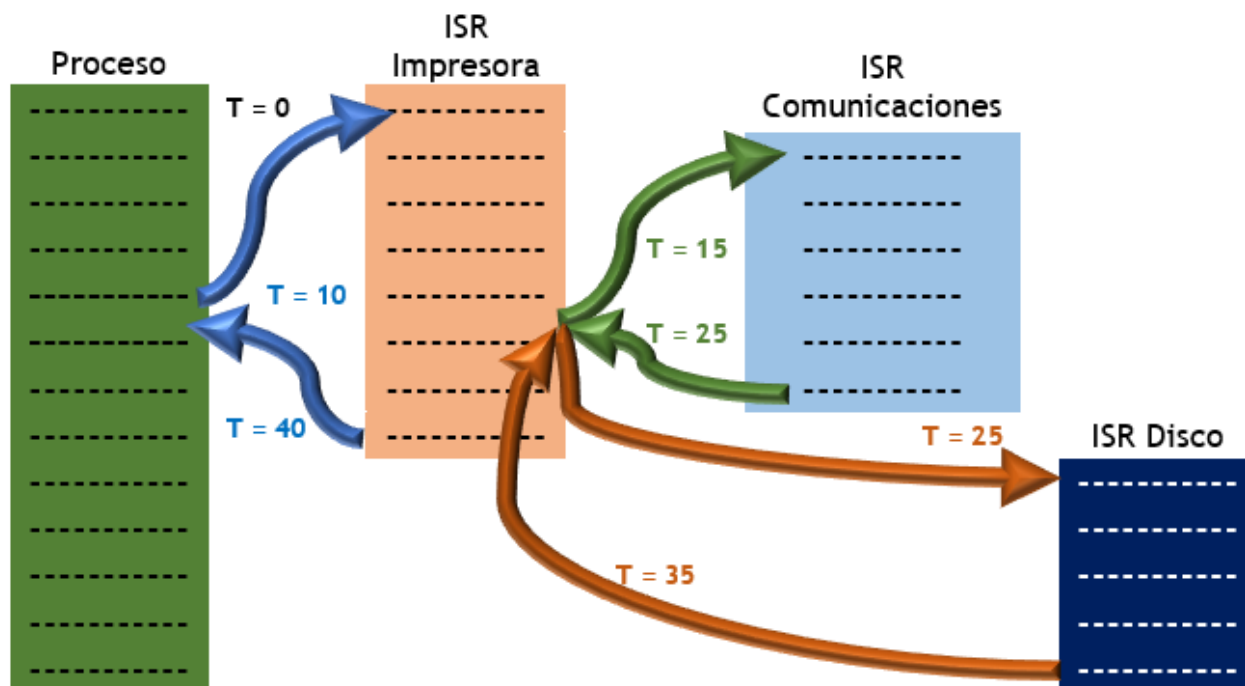
Observemos que es importante almacenar toda la información del estado del programa interrumpido para que éste pueda reanudarse. Esto se debe a que la interrupción no es una llamada a una rutina realizada desde el programa. En cambio, la interrupción puede producirse en cualquier momento y, por consiguiente, en cualquier punto de la ejecución del programa de usuario. Una interrupción es impredecible. De hecho, los dos programas pueden no tener nada en común y pueden pertenecer a distintos usuarios.

Hasta ahora únicamente se ha discutido la existencia de una sola interrupción. Supóngase, no obstante, que se pueden producir varias interrupciones. Por ejemplo, un programa puede estar recibiendo datos a través de una línea de comunicación o imprimiendo resultados. La impresora generará interrupciones cada vez que complete una operación de escritura. El controlador de la línea de comunicación generará una interrupción cada vez que llegue una unidad de datos. La unidad de datos puede ser un carácter o un bloque, según el protocolo de comunicación. En cualquier caso, es posible que se produzca una interrupción de comunicaciones mientras se está procesando la interrupción de la impresora. Se pueden seguir dos alternativas para tratar las interrupciones múltiples.

La primera es desactivar las interrupciones mientras se está procesando una interrupción. Una interrupción inhabilitada (disabled Interrupt) simplemente significa que el procesador puede y debe ignorar la señal de petición de interrupción. Si se produce una interrupción en ese momento, la misma generalmente se mantendrá pendiente y será examinada por el procesador una vez éste haya habilitado las interrupciones. Así, cuando un programa de usuario se está ejecutando y se produce una interrupción, las interrupciones se inhabilitan inmediatamente. Después de que la rutina de gestión de interrupción termine, las interrupciones se habilitan antes de que el programa de usuario prosiga, y el procesador comprueba si se han producido interrupciones adicionales. Esta aproximación es correcta y simple, puesto que las interrupciones se manejan en un orden secuencial estricto, pero su inconveniente es que no tiene en cuenta la prioridad relativa ni las solicitudes con un tiempo crítico. Por ejemplo, cuando llega una entrada desde la línea de comunicaciones, ésta debe absorberse rápidamente para dejar espacio a los datos siguientes. Si los primeros datos no se han procesado antes de que lleguen los siguientes, se pueden perder.

Una segunda alternativa consiste en definir prioridades para las interrupciones y permitir que una interrupción de prioridad más alta pueda interrumpir a un gestor de interrupción de prioridad menor. Como ejemplo de esta segunda alternativa, considérese un sistema con tres dispositivos de E/S: una impresora, un disco, y una línea de comunicaciones, con prioridades crecientes de 2, 4, y 5 respectivamente.

La figura siguiente muestra una posible secuencia. Un proceso de usuario comienza en $t=0$. En $t=10$, la impresora produce una interrupción; la información del proceso de usuario se sitúa en la pila del sistema, y la ejecución continúa con la rutina de servicio de interrupción de la impresora. Mientras se está ejecutando esta rutina, en $t=15$, se produce una interrupción de comunicaciones. Como la línea de comunicaciones tiene una prioridad mayor que la impresora, se acepta la interrupción. La rutina de la impresora se interrumpe, su estado se introduce en la pila, y la ejecución continúa con la rutina de comunicaciones. Mientras se está ejecutando esta rutina, el disco ocasiona una interrupción ($t=20$).



Puesto que esta interrupción tiene una prioridad menor, simplemente se retiene, y la rutina de comunicaciones se ejecuta hasta que termina. Cuando la rutina de comunicaciones termina ($t=25$), se restaura el estado previo del procesador, que corresponde a la ejecución de la rutina de la impresora. No obstante, antes incluso de que pueda ejecutarse una sola instrucción de esa rutina, el procesador acepta la interrupción, de mayor prioridad, generada por el disco y el control se transfiere a la rutina de disco. Sólo cuando esta rutina termina ($t=35$) la rutina de la impresora puede reanudarse. Cuando termina esa rutina ($t=40$), el control vuelve finalmente al proceso de usuario.

Jerarquía de memoria

La CPU sólo puede cargar instrucciones desde la memoria, por lo que cualquier programa debe cargarse primero en la memoria para ejecutarse. Los computadores de propósito general ejecutan la mayoría de sus programas desde una memoria regrabable, llamada memoria principal, también llamada memoria de acceso aleatorio o RAM. La memoria principal suele estar implementada en una tecnología de semiconductores llamada memoria dinámica de acceso aleatorio (DRAM).

Los computadores también utilizan otras formas de memoria. Por ejemplo, como ya se mencionó, el primer programa que se ejecuta al encender el computador es un programa de arranque, que luego carga el sistema operativo. Como la memoria RAM es volátil (pierde su contenido cuando se apaga o se pierde la energía) no podemos confiar en ella para mantener

el programa de arranque. En su lugar, para este y otros propósitos, el computador utiliza una memoria programable de sólo lectura que se puede borrar eléctricamente (EEPROM) y otras formas de firmware (almacenamiento que se escribe con poca frecuencia y que no es volátil). La EEPROM puede modificarse, pero no puede cambiarse con frecuencia. Además, es de baja velocidad, por lo que contiene principalmente programas y datos estáticos que no se utilizan con frecuencia. Por ejemplo, el iPhone utiliza la EEPROM para almacenar los números de serie y la información de hardware del dispositivo.

Todas las formas de memoria proporcionan una matriz de bytes. Cada byte tiene su propia dirección. La interacción se consigue mediante una secuencia de instrucciones de carga o almacenamiento a direcciones de memoria específicas. La instrucción de carga mueve un byte o una palabra de la memoria principal a un registro interno de la CPU, mientras que la instrucción de almacenamiento mueve el contenido de un registro a la memoria principal. Aparte de las cargas y almacenamientos explícitos, la CPU carga automáticamente las instrucciones de la memoria principal para su ejecución desde la ubicación almacenada en el contador de programa.

Un ciclo típico de ejecución de instrucciones, tal y como se ejecuta en un sistema con arquitectura von Neumann, primero obtiene una instrucción de la memoria y la almacena en el registro de instrucciones. A continuación, la instrucción se descodifica y puede hacer que los operandos se obtengan de la memoria y se almacenen en algún registro interno. Una vez ejecutada la instrucción sobre los operandos, el resultado puede volver a almacenarse en la memoria. Observemos que la unidad de memoria sólo ve un flujo de direcciones de memoria. No sabe cómo se generan (por el contador de instrucciones, la indexación, la indirección, las direcciones literales o algún otro medio) o para qué son (instrucciones o datos). Por lo tanto, podemos ignorar cómo se genera una dirección de memoria en un programa. Sólo nos interesa la secuencia de direcciones de memoria generadas por el programa en ejecución.

Idealmente, queremos que los programas y los datos residan en la memoria principal de forma permanente. Este arreglo generalmente no es posible en la mayoría de los sistemas por dos razones:

- 1.- La memoria principal suele ser demasiado pequeña para almacenar todos los programas y datos necesarios de forma permanente.
- 2.- La memoria principal, como ya se ha dicho, es volátil: pierde su contenido cuando se apaga o se pierde.

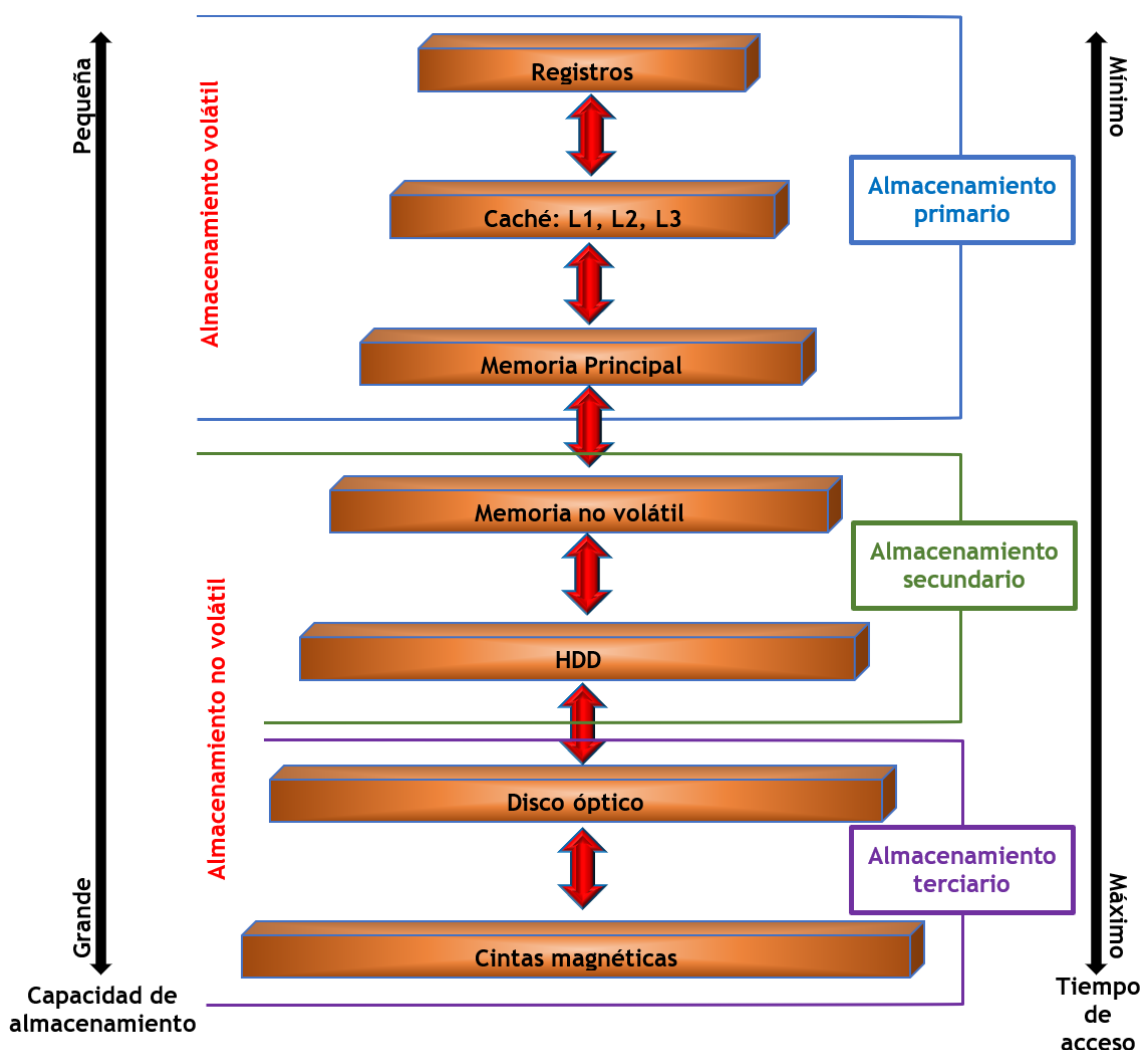
Por lo tanto, la mayoría de los sistemas de computación proporcionan un almacenamiento secundario como una extensión de la memoria principal. El principal requisito para el almacenamiento secundario es que sea capaz de mantener grandes cantidades de datos de forma permanente.

Los dispositivos de almacenamiento secundario más comunes son las unidades de disco duro (HDD) y los dispositivos de memoria no volátil, que proporcionan almacenamiento tanto para programas como para datos. La mayoría de los programas (de sistema y de aplicación) se almacenan en el almacenamiento secundario hasta que se cargan en la memoria. Muchos programas utilizan entonces el almacenamiento secundario como fuente y destino de su procesamiento. El almacenamiento secundario también es mucho más lento que la memoria

principal. Por lo tanto, la gestión adecuada del almacenamiento secundario es de vital importancia para un sistema de computación.

En un sentido más amplio, sin embargo, la estructura de almacenamiento que hemos descrito, consistente en registros, memoria principal y almacenamiento secundario, es sólo uno de los muchos diseños posibles de sistemas de almacenamiento. Otros componentes posibles son la memoria caché, el CD-ROM o Blu-ray, las cintas magnéticas, etc. Aquellos que son lo suficientemente lentos y grandes como para que sean utilizados sólo con fines especiales, como almacenar copias de seguridad de material almacenado en otros dispositivos, se denominan almacenamiento terciario. Cada uno de estos sistemas de almacenamiento proporciona las funciones básicas de almacenar un dato y mantenerlo hasta que se recupere en un momento posterior. Las principales diferencias entre los distintos sistemas de almacenamiento radican en la velocidad, el tamaño y la volatilidad.

La gran variedad de sistemas de almacenamiento puede organizarse en una jerarquía según la capacidad de almacenamiento y el tiempo de acceso. Por regla general, existe un equilibrio entre el tamaño y la velocidad, con una memoria más pequeña y rápida cerca de la CPU. Como se muestra en la figura, además de diferir en velocidad y capacidad, los distintos sistemas de almacenamiento son volátiles o no volátiles. El almacenamiento volátil, como ya se ha mencionado, pierde su contenido cuando se desconecta la alimentación del dispositivo, por lo que los datos deben escribirse en el almacenamiento no volátil para su conservación.



Los cuatro niveles superiores de memoria de la figura están contruidos con memoria semiconductora, que consiste en circuitos electrónicos basados en semiconductores. Los dispositivos de almacenamiento no volátil, en el cuarto nivel, tienen diversas variantes, pero en general son más rápidos que los discos duros. La forma más común de dispositivo de almacenamiento no volátil es la memoria flash, muy popular en dispositivos móviles como smartphones, tablets y SSD. Cada vez más, la memoria flash se utiliza también para el almacenamiento a largo plazo en portátiles, computadoras de escritorio y servidores.

Dado que el almacenamiento desempeña un papel importante en la estructura del sistema operativo, en este punto resulta conveniente aclarar el significado de diferentes términos empleados muy habitualmente. Repasemos la siguiente terminología:

- El almacenamiento volátil usualmente referenciado como memoria.
- El almacenamiento no volátil conserva su contenido cuando se pierde la alimentación. La mayor parte de los textos dedicados al almacenamiento secundario están referidos a este tipo de almacenamiento. Este tipo de almacenamiento puede clasificarse en dos tipos distintos:
 - Mecánico. Algunos ejemplos de estos sistemas de almacenamiento son los discos duros, los discos ópticos, el almacenamiento holográfico y la cinta magnética.
 - Eléctrico. Algunos ejemplos de este tipo de sistemas de almacenamiento son la memoria flash, la FRAM, la NRAM y los SSD.

El almacenamiento mecánico suele ser más grande y menos costoso por byte que el eléctrico. Por el contrario, el almacenamiento eléctrico suele ser más costoso, más pequeño y rápido que el mecánico.

El diseño de un sistema de almacenamiento completo debe equilibrar todos los factores que acabamos de comentar: debe utilizar sólo la cantidad de memoria cara que sea necesaria y, al mismo tiempo, proporcionar la mayor cantidad posible de almacenamiento barato y no volátil. Las cachés pueden instalarse para mejorar el rendimiento cuando existe una gran disparidad en el tiempo de acceso o la velocidad de transferencia entre dos componentes.

Elementos para las operaciones de entrada / salida

Una gran parte del código del sistema operativo se dedica a gestionar la E/S, tanto por su importancia para la fiabilidad y el rendimiento de un sistema como por la naturaleza variable de los dispositivos.

Recordemos que un sistema informático de propósito general consiste en múltiples dispositivos, todos los cuales intercambian datos a través de un bus común. La forma de E/S controlada mediante interrupciones descrita previamente, está bien para mover pequeñas cantidades de datos, pero puede producir una alta sobrecarga cuando se utiliza para el movimiento de datos a gran escala como una E/S desde o hacia una unidad de almacenamiento no volátil. Para resolver este problema, se utiliza el acceso directo a memoria (DMA). Después de configurar búferes, punteros y contadores para el dispositivo de E/S, el controlador del dispositivo transfiere un bloque entero de datos directamente hacia o desde el dispositivo y la memoria principal, sin intervención de la CPU. Sólo se genera una interrupción por bloque, para indicar al controlador de dispositivo que la operación ha finalizado, en lugar de la interrupción por byte que se genera con los dispositivos de baja

velocidad. Mientras el controlador del dispositivo realiza estas operaciones, la CPU está disponible para realizar otras tareas.

Algunos sistemas de gama alta utilizan la arquitectura de conmutación en lugar de la de bus. En estos sistemas, varios componentes pueden hablar con otros componentes simultáneamente, en lugar de competir por ciclos en un bus compartido. En este caso, el DMA es aún más eficaz.

Bibliografía de referencia del texto e imágenes

- Carretero Pérez, J., De Miguel Anasagasti, P., García Carballeira, F., & Pérez Costoya, F. (2007). *Sistemas Operativos: Una visión aplicada, Segunda Edición*. Buenos Aires: Mc. Graw Hill.
- Carretero Perez, J., Garcia Carballeira, F., & Perez Costoya, F. (2021). *Sistemas Operativos: Una Visión Aplicada, Tercera Edición, Volumen I*. Independently Published.
- Carretero Perez, J., Garcia Carballeira, F., & Perez Costoya, F. (2021). *Sistemas Operativos: Una Visión Aplicada, Tercera Edición, Volumen II*. Independently Published.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts, Tenth Edition*. Hoboken, New Jersey: John Wiley & Sons, Inc.
- Silva, M. (2015). *Sistemas Operativos*. Ciudad Autónoma de Buenos Aires: Alfaomega Grupo Editor.
- Stallings, W. (2012). *Operating Systems, Internals and Design Principles, Seventh Edition*. Upper Saddle River; New Jersey: Pearson Education, Inc.
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles, 9th Edition*. Upper Saddle River; New Jersey: Pearson Education, Inc.
- Tanenbaum, A. S. (2009). *Sistemas operativos modernos Tercera edición*. México: Pearson/Prentice Hall.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems, Fourth Edition*. Upper Saddle River, New Jersey: Pearson Education, Inc.
- Wolf, G. (2015). *Fundamentos de sistemas operativos*. México: Instituto de Investigaciones Económicas UNAM.